coralME Documentation

Release v1.2.3.1-17-g1749a80

Juan D. Tibocha Bonilla, Rodrigo Santibanez-Palominos

Jun 15, 2026

CONTENTS

CHAPTER

ONE

DESCRIPTION

The **COmprehensive Reconstruction ALgorithm for ME-models (coralME)** is an automatic pipeline for the reconstruction of ME-models. coralME integrates existing ME-modeling packages COBRAme, ECOLIme, and solveME, generalizes their functions for implementation on any prokaryote, and processes readily available organism-specific inputs for the automatic generation of a working ME-model.

coralME has four main objectives:

1. **Synchronize** input files to remove contradictory entries.
2. **Complement** input files from homology with a template organism to complete the E-matrix.
3. **Build** a working ME-model
4. **Inform** the user about necessary steps to curate the ME-model.

This resource is intended to:

1. Describe basic inputs required for ME-model reconstruction.
2. Describe the architecture of coralME.
3. Demonstrate how to build a ME-model with coralME.
4. Describe how to perform manual curation guided by coralME's curation notes.

CHAPTER

TWO

CONTENT

2.1 Getting started

2.1.1 Reconstruct with coralME

Here we show an example to reconstruct a dME-model of *B. subtilis*.

Import packages

[1]:
```
from coralme.builder.main import MEBuilder
from importlib.resources import files
```

Define path to inputs

Define the paths to inputs and desired outputs. For more information about these files see *Description of inputs*.

[2]:
```
input = {
  # Inputs
  "m-model-path": "./helper_files/inputs/m_model.json", # Path to model file
  "genbank-path": "./helper_files/inputs/genome.gb", # Path to genome genbank file

  # Outputs
  "df_gene_cplxs_mods_rxns": "./helper_files/building_data/OSM.xlsx", # Desired output␣
↪path of OSM
  "out_directory": "./helper_files/", # Output directory
  "log_directory": "./helper_files/", # Log directory
  "locus_tag": "old_locus_tag", # What IDs were used in the M-model? e.g. locus_tag, old_
↪locus_tag
  "ME-Model-ID" : "EXAMPLE-BACILLUS-ME" # Name of the ME-model
}
```

Load organism setup. For now we can use the minimal setup with biological numbers from E. coli.

[3]:
```
organism = str(files("coralme") / "iJL1678b") + "-ME/minimal-organism.json"
```

Create builder

For more information about this class see *Architecture of coralME*

[4]:
```
builder = MEBuilder(*[organism], **input)
```

Generate files

This corresponds to *Synchronyze* and *Complement* steps in *Architecture of coralME*

[5]:
```
builder.generate_files(overwrite=True)
```

```
Initiating file processing...
~ Processing files for EXAMPLE-BACILLUS-ME...
Set parameter Username
Academic license - for non-commercial use only - expires 2025-09-03
```

```
Checking M-model metabolites...                                    : 100.0%||  ␣
↪990/  990 [00:00<00:00]
Checking M-model genes...                                          : 100.0%||  ␣
↪844/  844 [00:00<00:00]
Checking M-model reactions...                                      : 100.0%|| ␣
↪1250/ 1250 [00:00<00:00]
Syncing optional genes file...                                     : 0.0%|     ␣
↪    |     0/    0 [00:00<?]
Looking for duplicates within datasets...                          : 100.0%||  ␣
↪ 5/    5 [00:00<00:00]
Gathering ID occurrences across datasets...                        : 100.0%|| ␣
↪1250/ 1250 [00:00<00:00]
Solving duplicates across datasets...                              : 0.0%|     ␣
↪    |     0/    0 [00:00<?]
Pruning GenBank...                                                 : 100.0%||  ␣
↪ 1/    1 [00:01<00:00]
Updating Genbank file with optional files...                       : 0.0%|     ␣
↪    |     0/    0 [00:00<?]
Syncing optional files with genbank contigs...                     : 100.0%||  ␣
↪ 1/    1 [00:08<00:00]
Modifying metabolites with manual curation...                      : 0.0%|     ␣
↪    |     0/    0 [00:00<?]
Modifying metabolic reactions with manual curation...              : 0.0%|     ␣
↪    |     0/    0 [00:00<?]
Adding manual curation of complexes...                             : 0.0%|     ␣
↪    |     0/    0 [00:00<?]
Getting sigma factors...                                           : 100.0%||  ␣
↪17/   17 [00:00<00:00]
Getting generics from Genbank contigs...                           : 100.0%||  ␣
↪ 1/    1 [00:00<00:00]
Getting TU-gene associations from optional TUs file...             : 0.0%|     ␣
↪    |     0/    0 [00:00<?]
Gathering ribosome stoichiometry...                                : 100.0%||  ␣
↪63/   63 [00:00<00:00]
Adding protein location...                                         : 100.0%|| ␣
↪4238/ 4238 [00:00<00:00]
Purging M-model genes...                                           : 100.0%||  ␣
↪844/  844 [00:00<00:00]
Getting enzyme-reaction associations...                            : 100.0%|| ␣
↪1250/ 1250 [00:04<00:00]
```

```
Reading EXAMPLE-BACILLUS-ME done.
```

```
Gathering M-model compartments...                                  : 100.0%||  ␣
↪ 2/    2 [00:00<00:00]
```

(continues on next page)

(continued from previous page)

```
Fixing compartments in M-model metabolites...                                  : 100.0%||  ␣
↪990/  990 [00:00<00:00]
Fixing missing names in M-model reactions...                                   : 100.0%|| ␣
↪1250/ 1250 [00:00<00:00]
Updating enzyme reaction association...                                        : 100.0%||  ␣
↪902/  902 [00:00<00:00]
Getting tRNA to codon dictionary from NC_000964.3                              : 100.0%|| ␣
↪4449/ 4449 [00:03<00:00]
Checking defined translocation pathways...                                     : 0.0%|     ␣
↪    |     0/    0 [00:00<?]
Getting reaction Keffs...                                                      : 100.0%||  ␣
↪902/  902 [00:00<00:00]
```

```
Writting the Organism-Specific Matrix...
Organism-Specific Matrix saved to ./helper_files/building_data/OSM.xlsx file.
File processing done.
```

Build ME-model

This corresponds to *Build* in *Architecture of coralME*

[6]:
```
builder.build_me_model(overwrite=False)
```

```
Initiating ME-model reconstruction...
```

```
Adding biomass constraint(s) into the ME-model...                              : 100.0%||  ␣
↪11/   11 [00:00<00:00]
```

```
Read LP format model from file /tmp/tmp6wx5hvsz.lp
Reading time = 0.00 seconds
: 990 rows, 2500 columns, 10478 nonzeros
```

```

```

```
Read LP format model from file /tmp/tmp053a6_ir.lp
Reading time = 0.00 seconds
: 990 rows, 2496 columns, 10342 nonzeros
```

```
Adding Metabolites from M-model into the ME-model...                           : 100.0%||  ␣
↪990/  990 [00:00<00:00]
Adding Reactions from M-model into the ME-model...                             : 100.0%|| ␣
↪1248/ 1248 [00:00<00:00]
Adding Transcriptional Units into the ME-model from user input...              : 0.0%|     ␣
↪    |     0/    0 [00:00<?]
Adding features from contig NC_000964.3 into the ME-model...                   : 100.0%|| ␣
↪4449/ 4449 [00:08<00:00]
Updating all TranslationReaction and TranscriptionReaction...                  : 100.0%|| ␣
↪9232/ 9232 [00:21<00:00]
Removing SubReactions from ComplexData...                                      : 100.0%|| ␣
↪4340/ 4340 [00:00<00:00]
Adding ComplexFormation into the ME-model...                                   : 100.0%|| ␣
↪4340/ 4340 [00:00<00:00]
Adding Generic(s) into the ME-model...                                         : 100.0%||  ␣
↪ 5/    5 [00:00<00:00]
Processing StoichiometricData in ME-model...                                   : 100.0%|| ␣
↪1020/ 1020 [00:00<00:00]
```

```
ME-model was saved in the ./helper_files/ directory as MEModel-step1-EXAMPLE-BACILLUS-ME.
↪pkl
Adding tRNA synthetase(s) information into the ME-model...                : 100.0%||  ␣
↪306/  306 [00:00<00:00]
Adding tRNA modification SubReactions...                                  : 0.0%|     ␣
↪    |     0/    0 [00:00<?]
Associating tRNA modification enzyme(s) to tRNA(s)...                     : 0.0%|     ␣
↪    |     0/    0 [00:00<?]
Adding SubReactions into TranslationReactions...                          : 100.0%|| ␣
↪4238/ 4238 [00:00<00:00]
Adding RNA Polymerase(s) into the ME-model...                             : 100.0%||  ␣
↪17/   17 [00:00<00:00]
Associating a RNA Polymerase to each Transcriptional Unit...              : 0.0%|     ␣
↪    |     0/    0 [00:00<?]
Processing ComplexData in ME-model...                                     : 100.0%||  ␣
↪ 1/    1 [00:00<00:00]
Adding ComplexFormation into the ME-model...                              : 100.0%|| ␣
↪4365/ 4365 [00:00<00:00]
Adding SubReactions into TranslationReactions...                          : 100.0%|| ␣
↪4238/ 4238 [00:00<00:00]
Adding Transcription SubReactions...                                      : 100.0%|| ␣
↪4449/ 4449 [00:00<00:00]
Processing StoichiometricData in SubReactionData...                       : 0.0%|     ␣
↪    |     0/    0 [00:00<?]
Adding reaction subsystems from M-model into the ME-model...              : 100.0%|| ␣
↪1248/ 1248 [00:00<00:00]
Processing StoichiometricData in ME-model...                              : 100.0%|| ␣
↪1020/ 1020 [00:00<00:00]
Updating ME-model Reactions...                                            : 100.0%||␣
↪16069/16069 [01:11<00:00]
Updating all FormationReactions...                                        : 100.0%|| ␣
↪4365/ 4365 [00:00<00:00]
Recalculation of the elemental contribution in SubReactions...            : 100.0%||  ␣
↪82/   82 [00:00<00:00]
Updating all FormationReactions...                                        : 100.0%|| ␣
↪4365/ 4365 [00:00<00:00]
Updating FormationReactions involving a lipoyl prosthetic group...        : 0.0%|     ␣
↪    |     0/    0 [00:00<?]
Updating FormationReactions involving a glycyl radical...                 : 0.0%|     ␣
↪    |     0/    0 [00:00<?]
Estimating effective turnover rates for reactions using the SASA method... : 100.0%|| ␣
↪2658/ 2658 [00:00<00:00]
Mapping effective turnover rates from user input...                       : 0.0%|     ␣
↪    |     0/    0 [00:00<?]
Setting the effective turnover rates using user input...                  : 100.0%|| ␣
↪2424/ 2424 [00:00<00:00]
Pruning unnecessary ComplexData reactions...                              : 100.0%|| ␣
↪4365/ 4365 [00:04<00:00]
Pruning unnecessary FoldedProtein reactions...                            : 0.0%|     ␣
↪    |     0/    0 [00:00<?]
Pruning unnecessary ProcessedProtein reactions...                         : 100.0%|| ␣
↪4239/ 4239 [00:00<00:00]
```

(continues on next page)

(continued from previous page)

```
Pruning unnecessary TranslatedGene reactions...                    : 100.0%|| ␣
↪4239/ 4239 [00:08<00:00]
Pruning unnecessary TranscribedGene reactions...                   : 100.0%|| ␣
↪4450/ 4450 [00:04<00:00]
Pruning unnecessary Transcriptional Units...                       : 100.0%|| ␣
↪4449/ 4449 [00:03<00:00]
Pruning unnecessary ComplexData reactions...                       : 100.0%|| ␣
↪776/  776 [00:00<00:00]
Pruning unnecessary FoldedProtein reactions...                     : 0.0%|    ␣
↪    |     0/    0 [00:00<?]
Pruning unnecessary ProcessedProtein reactions...                  : 100.0%|| ␣
↪948/  948 [00:00<00:00]
Pruning unnecessary TranslatedGene reactions...                    : 100.0%|| ␣
↪948/  948 [00:00<00:00]
Pruning unnecessary TranscribedGene reactions...                   : 100.0%|| ␣
↪1064/ 1064 [00:01<00:00]
Pruning unnecessary Transcriptional Units...                       : 100.0%|| ␣
↪1063/ 1063 [00:01<00:00]
Pruning unnecessary ComplexData reactions...                       : 100.0%|| ␣
↪760/  760 [00:00<00:00]
Pruning unnecessary FoldedProtein reactions...                     : 0.0%|    ␣
↪    |     0/    0 [00:00<?]
Pruning unnecessary ProcessedProtein reactions...                  : 100.0%|| ␣
↪932/  932 [00:00<00:00]
Pruning unnecessary TranslatedGene reactions...                    : 100.0%|| ␣
↪932/  932 [00:00<00:00]
Pruning unnecessary TranscribedGene reactions...                   : 100.0%|| ␣
↪1048/ 1048 [00:01<00:00]
Pruning unnecessary Transcriptional Units...                       : 100.0%|| ␣
↪1047/ 1047 [00:01<00:00]
```

```
ME-model was saved in the ./helper_files/ directory as MEModel-step2-EXAMPLE-BACILLUS-ME.
↪pkl
ME-model reconstruction is done.
Number of metabolites in the ME-model is 3808 (+284.65%, from 990)
Number of reactions in the ME-model is 7031 (+462.48%, from 1250)
Number of genes in the ME-model is 1046 (+23.93%, from 844)
Number of missing genes from reconstruction cannot be determined.
```

Troubleshoot ME-model

This corresponds to *Find gaps* in *Architecture of coralME*

[7]:
```
builder.troubleshoot(growth_key_and_value = { builder.me_model.mu : 0.001 })
```

```
The MINOS and quad MINOS solvers are a courtesy of Prof Michael A. Saunders. Please cite␣
↪Ma, D., Yang, L., Fleming, R. et al. Reliable and efficient solution of genome-scale␣
↪models of Metabolism and macromolecular Expression. Sci Rep 7, 40863 (2017). https://
↪doi.org/10.1038/srep40863

~ Troubleshooting started...
  Checking if the ME-model can simulate growth without gapfilling reactions...
  Original ME-model is not feasible with a tested growth rate of 0.001000 1/h
```

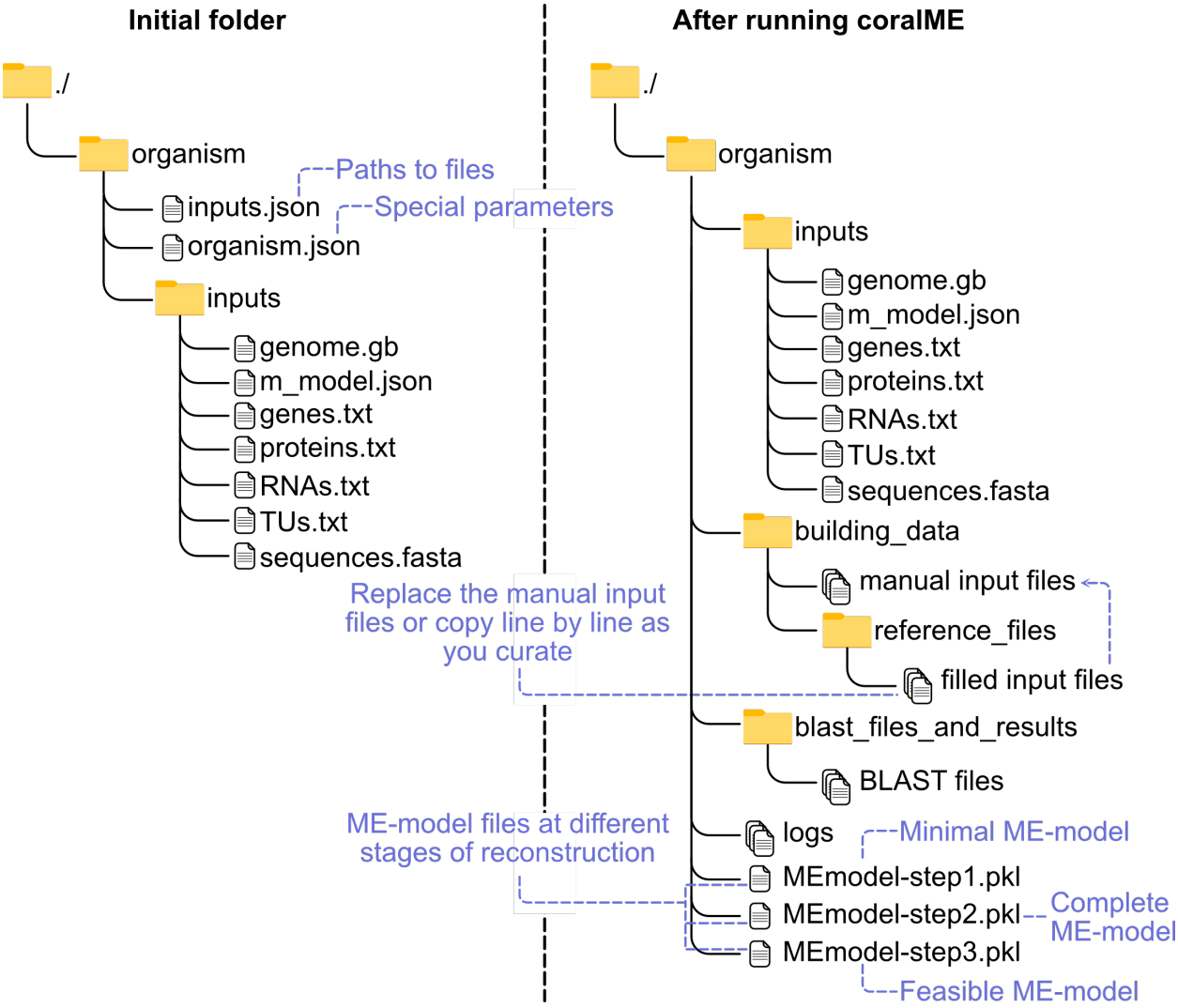
(continues on next page)

(continued from previous page)

```
  Step 1. Gapfill reactions to provide components of type 'ME-Deadends' using brute␣
↪force.
          Finding gaps in the ME-model...
          Finding gaps from the M-model only...
          0 metabolites were identified as deadends.
          Adding sink reactions for 0 metabolites...
          Provided metabolites through sink reactions cannot recover growth. Proceeding␣
↪to next set of metabolites.
  Step 2. Gapfill reactions to provide components of type 'Cofactors' using brute force.
          Adding sink reactions for 2 metabolites...
            Sink reactions shortlisted to 2 metabolites.
             Processed: 1/2, Gaps: 0. The ME-model is feasible if TS_mg2_c is closed.
             Processed: 2/2, Gaps: 1. The ME-model is not feasible if TS_zn2_c is closed.
~ Troubleshooter added the following sinks: TS_zn2_c.
~ Final step. Fully optimizing with precision 1e-6 and save solution into the ME-model...
  Gapfilled ME-model is feasible with growth rate 0.102178 (M-model: 0.117966).
ME-model was saved in the ./helper_files/ directory as MEModel-step3-EXAMPLE-BACILLUS-ME-
↪TS.pkl
```

2.1.2 Understanding the file structure

For a detailed explanation of the inputs, see For more information about these files see *Description of inputs*



2.1.3 Curate manually

1. **Copy** all of the generated *reference files* in building_data/reference_files and replace accordingly in building_data/
2. **Go one by one** through the files in building_data/ curating as needed! Important flags are risen in curation_notes.json to further guide you through curation.
3. Everytime you make a change, **run the model through the troubleshooter!** It will show you remaining gaps to look at, and the new curation notes might show new warnings.
4. **Keep iterating!** You will have finished when no gaps are present, and all remaining warnings in curation notes are irrelevant.

2.2 Description of Inputs

coralME takes a total of 7 inputs, 2 required and 5 optional. Additionally, it takes 2 configuration files.

2.2.1 Types of inputs

Required

1. **Genome file** (**genome.gb**)
2. **M-model** (**m_model.json** or **m_model.xml**)

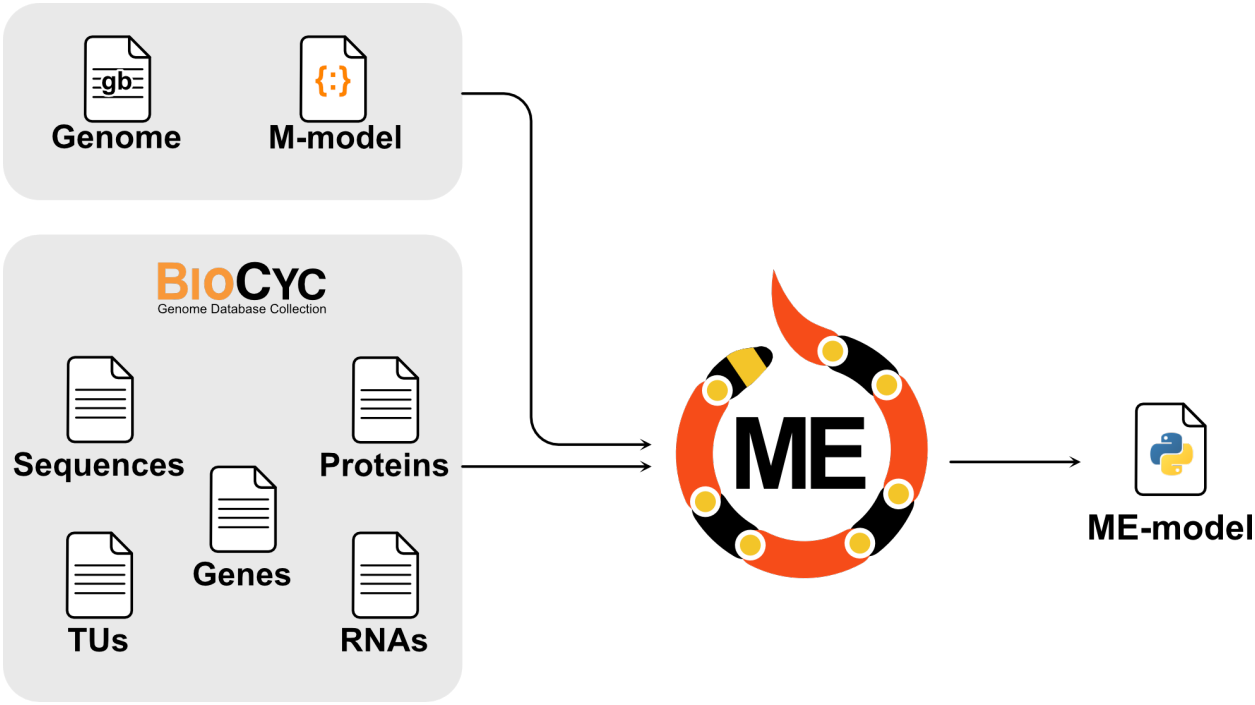
Optional

Downloadable from an existing **BioCyc** database under **Special SmartTables**. If no optional files are provided, coralME complements them with **genome.gb**

3. **Genes file**, by default: **genes.txt**
4. **RNAs file**, by default: **RNAs.txt**
5. **Proteins file**, by default: **proteins.txt**
6. **TUs file**, by default: **TUs.txt.**
7. **Sequences file**, by default: **sequences.fasta**

Configuration

8. **Paths file**, by default: **inputs.json**
9. **Parameters file**, by default: **organism.json**



2.2.2 Description

Genome (genome.gb)

Description

The genome file contains provides coralME with:

- Gene annotations.
- Gene sequences.

Requirements

1. Locus tags (locus_tag or old_locus_tag) MUST be consistent with **m_model.json**. Make sure you download the same genome file that was used to reconstruct the M-model.
2. Has name **genome.gb**.
3. Genbank-compliant file. Must be read by BioPython correctly.
4. It must contain the entire genome sequence. Make sure to enable **Customize View>Show Sequence** before downloading the genbank file from NCBI.

See an example of genome.gb and sequences.fasta

M-model (m_model.json)

Description

The M-model provides coralME with the metabolic model components:

- Metabolic network (M-matrix)
- Gene-protein-reaction associations
- Environmental and internal constraints
- Reaction subsystems
- Biomass composition

Requirements

1. Gene identifiers MUST be consistent with **genome.gb** locus_tag or old_locus_tag. Make sure you download the same genome file that was used to reconstruct the M-model.
2. Has name **m_model.json**.
3. COBRApy-compliant. Must be read by cobrapy-0.25.0.

See an example of m_model.json

Gene dictionary (genes.txt) [optional]

Description

genes.txt is a gene information table that can be downloaded from the **All genes of organism SmartTable** of the BioCyc database. Click **Export>to Spreadsheet File>frame IDs**. This file is optional and is meant to complement the information from **genome.gb** in case the latter is missing genes.

genes.txt provides coralME with:

- Gene locus tags
- Gene names
- Gene annotations
- Gene positions
- Gene products (protein, tRNA, etc.)

Requirements

1. Contains the index **Gene Name** and columns **Accession-1**, **Left-End-Position**, **Right-End-Position**, and **Product**.
2. **Accession-1** MUST be consistent with the gene IDs in the GPRs of **m_model.json** and with the locus_tag (or old_locus_tag) in **genome.gb**.
3. **Gene Name** is consistent with:
    - Column **Genes of polypeptide, complex, or RNA** of **proteins.txt**
    - Column **Gene** of **RNAs.txt**
    - Column **Genes of transcription unit** of **TUs.txt**
    - Gene identifiers in **sequences.fasta**
4. **Product** is consistent with:
    - Index of **proteins.txt**
    - Index of **RNAs.txt**
5. Must be tab-separated

See an example of genes.txt

Note: Requirement 2 regarding ID consistency should be directly met if the files are downloaded from the correct BioCyc database.

Note: Requirements 3, 4 and 5 regarding ID consistency should be directly met if the files are downloaded from the same BioCyc database.

Proteins (proteins.txt) [optional]

Description

proteins.txt is a protein complex information table that can be downloaded from the **All proteins of organism SmartTable** of the BioCyc database. Click **Export>to Spreadsheet File>frame IDs**. This file is optional and is meant to complement the information from **genome.gb**.

proteins.txt provides coralME with:

- Protein complex compositions

Requirements

1. Contains the index **(Proteins Complexes)** and columns **Common-Name**, **Genes of polypeptide, complex, or RNA**, and **Locations**.
2. **(Proteins Complexes)** is consistent with:
    - Column **Product** of **genes.txt**
3. **Genes of polypeptide, complex, or RNA** is consistent with:

- Index **Gene Name** of **genes.txt**

4. Must be tab-separated

See an example of proteins.txt

Note: Requirements 2, 3 and 4 regarding ID consistency should be directly met if the files are downloaded from the same BioCyc database.

RNAs (RNAs.txt) [optional]

Description

RNAs.txt is an RNA annotation table that can be downloaded from the **All RNAs of organism SmartTable** of the BioCyc database. Click **Export>to Spreadsheet File>frame IDs**. This file is optional and is meant to complement the information from **genome.gb**.

RNAs.txt provides coralME with:

- Genes annotated as RNA products (e.g. tRNA, rRNA, etc.)
- RNA gene annotations (e.g. amino acids - tRNA associations)

Requirements

1. Contains the index **(All-tRNAs Misc-RNAs rRNAs)** and columns **Common-Name**, and **Gene**
2. **(All-tRNAs Misc-RNAs rRNAs)** is consistent with:
    - Column **Product** of **genes.txt**
3. **Gene** is consistent with:
    - Index **Gene Name** of **genes.txt**
4. Must be tab-separated

See an example of RNAs.txt

Note: Requirements 2, 3 and 4 regarding ID consistency should be directly met if the files are downloaded from the same BioCyc database.

TUs (TUs.txt) [optional]

Description

TUs.txt is a transcription unit annotation table that can be downloaded from the **All TUs of organism SmartTable** of the BioCyc database. Click **Export>to Spreadsheet File>frame IDs**. This file is optional and is meant to complement the information from **genome.gb**.

TUs.txt provides coralME with:

- Co-transcribed genes (operons).
- Direction of transcription.

- TU IDs.

Requirements

1. Contains the index **Transcription-Units** and columns **Genes of transcription unit**, and **Direction**
2. **Genes of transcription unit** is consistent with:
    - Index **Gene Name** of **genes.txt**
3. Must be tab-separated

See an example of TUs.txt

Note: Requirements 2 and 3 regarding ID consistency should be directly met if the files are downloaded from the same BioCyc database.

Gene sequences (sequences.fasta) [optional]

Description

sequences.fasta is a nucleotide FASTA file that can be downloaded from the **All genes of organism SmartTable** of the BioCyc database. Click **Export>FASTA>Find sequences**. This file is optional and is meant to complement the information from **genome.gb** in case the latter is missing genes.

sequences.fasta provides coralME with:

- Gene sequences

Requirements

1. Gene identifiers are consistent with:
    - Index **Gene Name** of **genes.txt**
2. Must be tab-separated

See an example of sequences.fasta

Note: Requirements 1, 2 and 3 regarding ID consistency should be directly met if the files are downloaded from the same BioCyc database.

Configuration of paths to files (inputs.json)

Description

inputs.json is a JSON file containing paths to input files for coralME.

inputs.json provides coralME with:

- Paths to input files

Requirements

1. Must be JSON-compliant
2. Must contain paths to required files (M-model and Genome).
3. All defined files must exist.

See an example of input.json

Configuration of parameters (organism.json)

Description

organism.json is a JSON file containing paths to input files for coralME.

organism.json provides coralME with:

- ME-modeling parameters

Requirements

1. Must be JSON-compliant
2. Must contain the standard fields.

See an example of organism.json

2.3 Adding annotation from a BioCyc Database

For details on inputs go to *Description of Inputs*.

For information about coralME architecture go to *Architecture of coralME*.

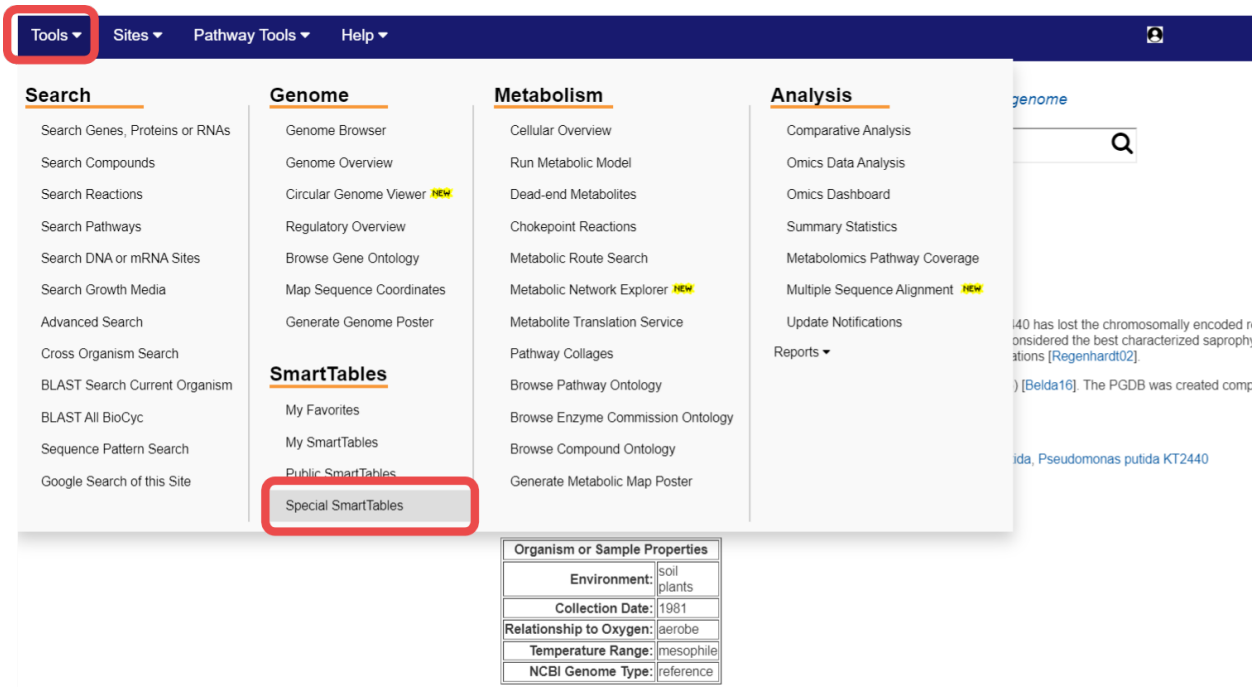
2.3.1 Download files from BioCyc

BioCyc files are optional but useful, you should download them after having your gene id consistent M-model and genbank files.

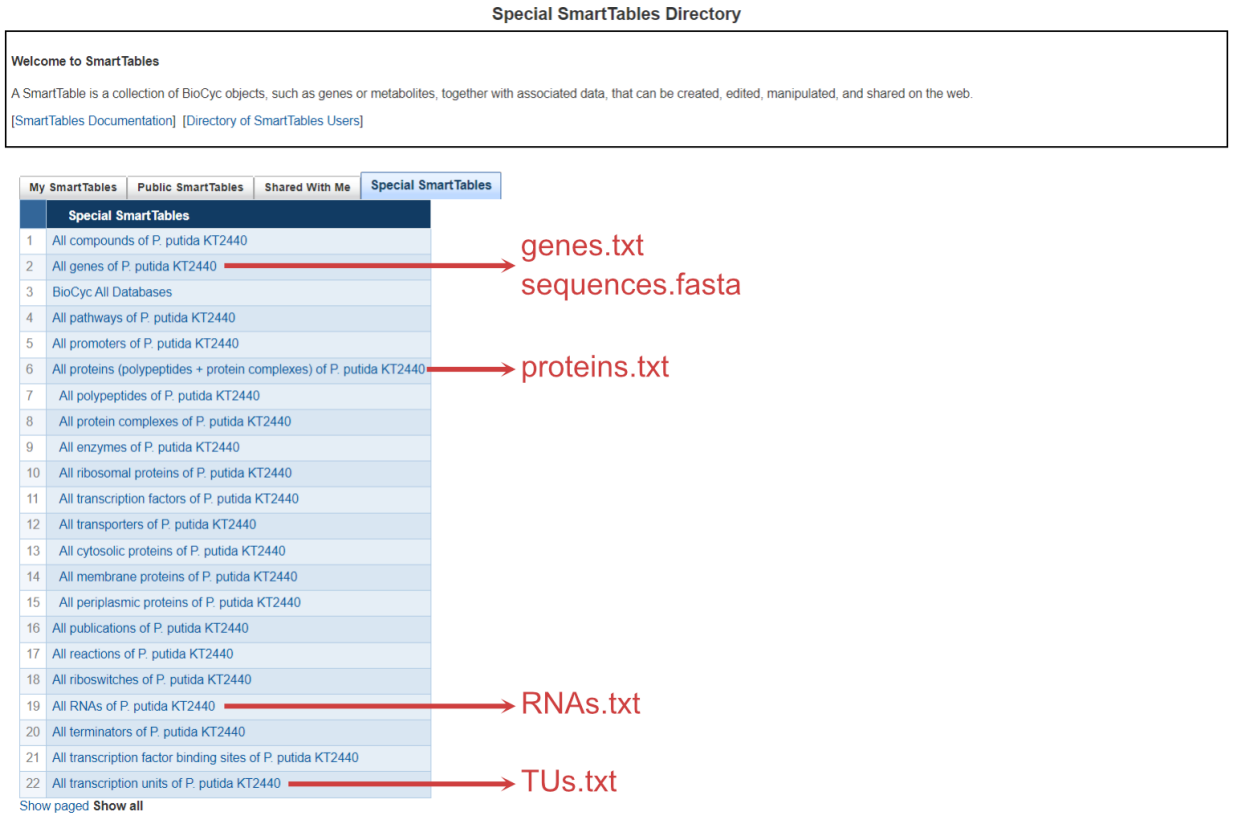
The quickest way to do this is to copy one of the genes from the genbank file into the BioCyc search bar. Your organism should appear in the list if it is available in BioCyc.

To download:

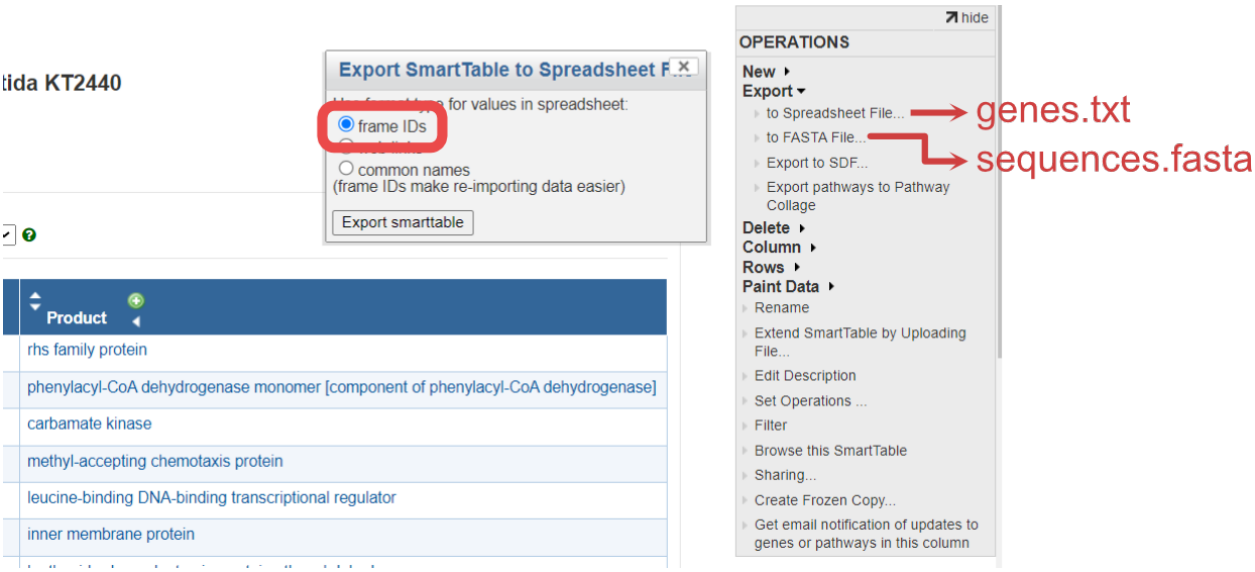
Go to Tools>Special SmartTables



From here you can download the 5 optional BioCyc files for your organism.



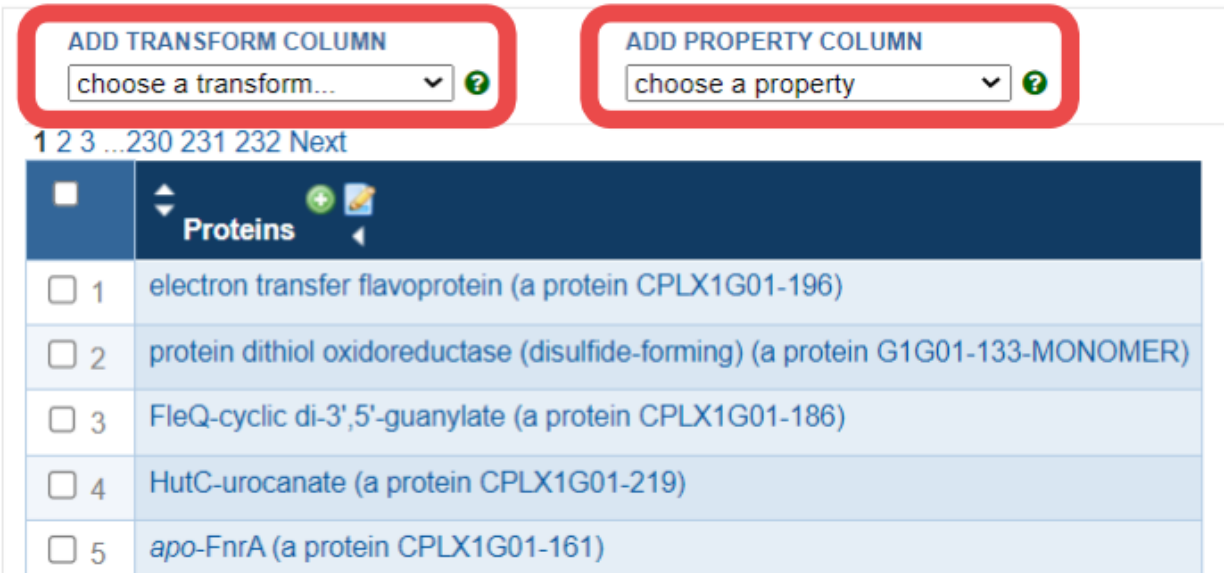
Download genes.txt and sequences.fasta



Download proteins.txt, RNAs.txt and TUs.txt

The same process of genes.txt applies to proteins.txt, RNAs.txt and TUs.txt.

Some columns must be added manually using BioCyc's dropdown lists **ADD PROPERTY COLUMN** and **ADD TRANSFORM COLUMN** within the SmartTable editing webpage.



Download proteins.txt

The index (Proteins Complexes) is in the SmartTable by default, but you need to add the columns Common-Name, Genes of polypeptide, complex, or RNA, and Locations.

- Common-Name is available in the dropdown list **ADD PROPERTY COLUMN**
- Genes of polypeptide, complex, or RNA is available in the dropdown list **ADD TRANSFORM COLUMN**

- Locations is available in the dropdown list **ADD PROPERTY COLUMN**.

Download RNAs.txt

The index (All-tRNAs Misc-RNAs rRNAs) is in the SmartTable by default, but you need to add the columns Common-Name, and Gene.

- Common-Name is available in the dropdown list **ADD PROPERTY COLUMN**
- Gene is available in the dropdown list **ADD PROPERTY COLUMN**.

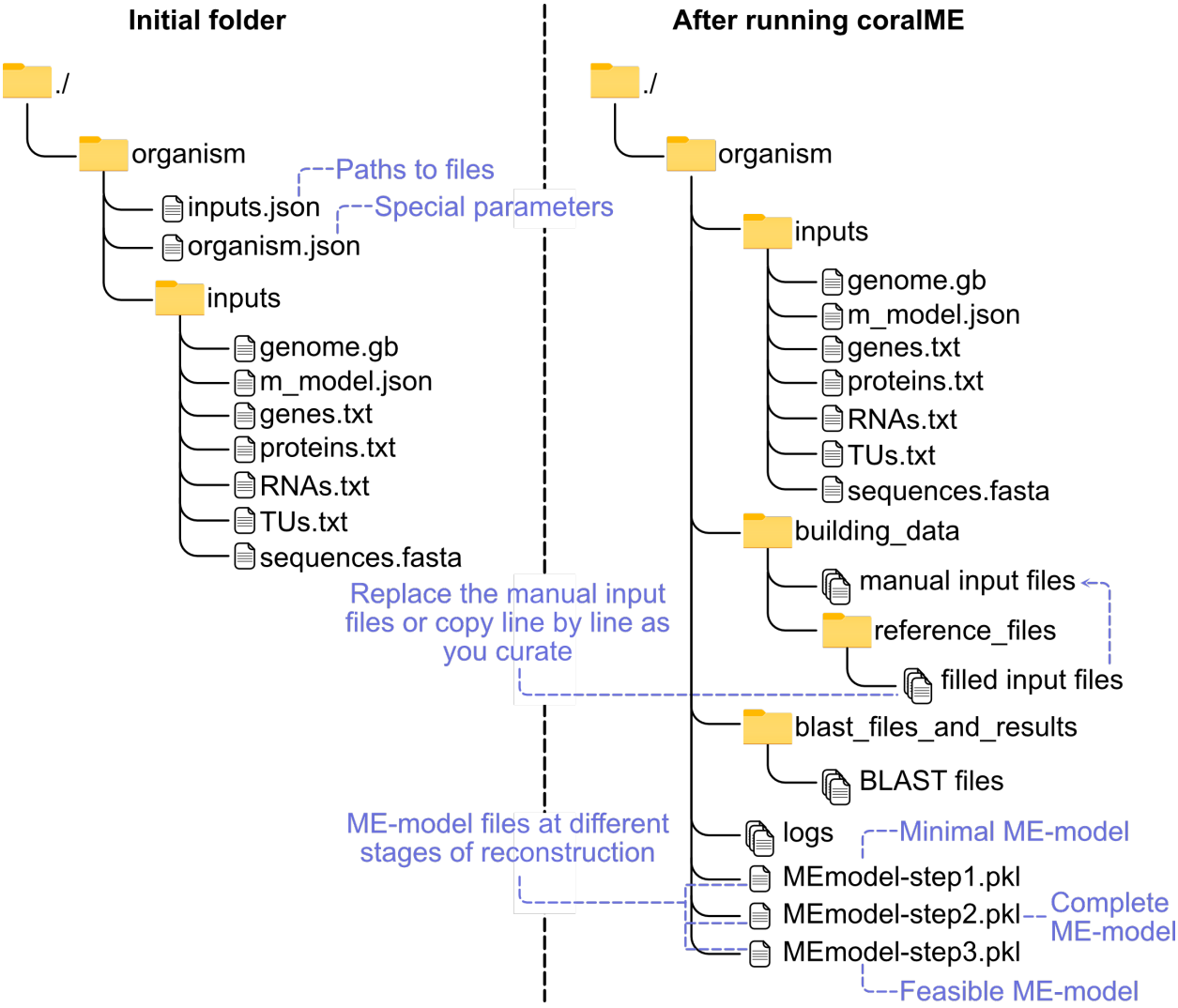
Download TUs.txt

The index Transcription-Units is in the SmartTable by default, but you need to add the columns Genes of transcription unit, and Direction.

- Genes of transcription unit is available in the dropdown list **ADD TRANSFORM COLUMN**
- Direction is available in the dropdown list **ADD PROPERTY COLUMN**.

2.3.2 Initialize the folder for your organism

Copy your files to create your initial folder



Define inputs in input.json.

See an example of input.json

Define parameters in organism.json.

See an example of organism.json

Note: You do not need to modify these parameters right away. But once you are at the curation stage you will have to ensure these parameters are applicable to your organism.

2.4 Architecture of coralME

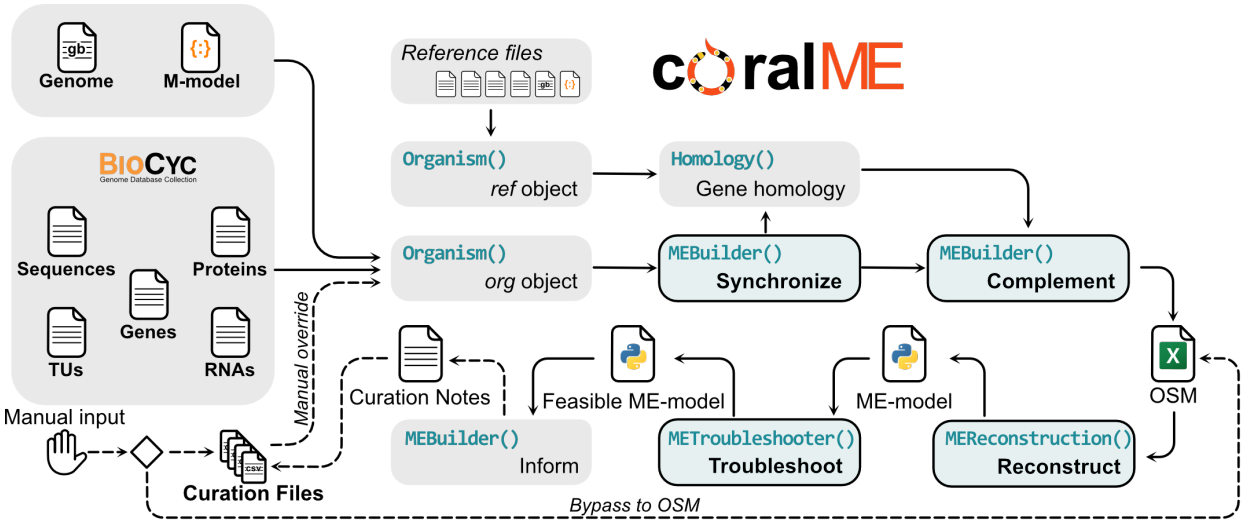
coralME is composed of 4 main classes that process and exchange organism-specific information for the reconstruction of a ME-model. The classes are:

```
class Organism()

class MEBuilder()

class MEReconstruction()

class Homology()
```



2.4.1 Organism()

class coralme.builder.organism.**Organism**(*config*, *is_reference*, *available_reference_models=None*)

Organism class for storing information about an organism

This class acts as a database containing all necessary information to reconstruct a ME-model. It is used to retrieve and store information of the main (org) and the reference (ref) organisms. Information in Organism is read and manipulated by methods in the MEBuilder class.

Parameters

- **config** (*dict*) – Dictionary containing configuration and settings.
- **is_reference** (*bool*) – If True, process as reference organism.

Role: Store information about an organism

This class acts as a database containing all necessary information to reconstruct a ME-model. It is used to retrieve and store information of the main (**org**) and the reference (**ref**) organisms. Information in Organism() is read and manipulated by methods in the MEBuilder() class. The reference can be set as any of the provided organisms in coralME, available here, although we advise to choose *E. coli* and *B. subtilis* for gram-negative and gram-positive bacteria, respectively.

2.4.2 MEBuilder()

class coralme.builder.main.**MEBuilder**(**args*, *m_model_path=None*, *genbank_path=None*, *locus_tag='locus_tag'*, *blast_threads='auto'*, *e_value_cutoff=1e-10*, *df_gene_cplxs_mods_rxns='automated-org-with-refs.xlsx'*, *df_TranscriptionalUnits=''*, *df_matrix_stoichiometry=''*, *df_matrix_subrxn_stoich=''*, *df_metadata_orphan_rxns=''*, *df_metadata_metabolites=''*, *out_directory='./'*, *log_directory='./'*, *parameter_set='stable'*, *estimate_keffs=True*, *add_lipoproteins=True*, *include_pseudo_genes=False*, *relax_rrna_modifications=False*, ***kwargs*)

MEBuilder class to coordinate the reconstruction of ME-models.

Parameters

- ***args** – Positional arguments are passed as paths to JSON files that update the configuration of the parent class.
- ****kwargs** – Further keyword arguments are passed on as dictionaries to update the configuration of the parent class.

Role: Coordinate the roles of other classes.

This class acts as the main coordinator between other objects, e.g. Organism, Homology, MEProcesser, and METroubleshooter. It contains methods to manipulate class Organism by using attributes in class Homology, and manually curated files in the folder containing the main organism. Moreover, it is called by objects to access stored information in other objects.

2.4.3 MEReconstruction()

class coralme.builder.main.**MEReconstruction**(*builder*)

MEReconstruction class for reconstructing a ME-model from user/automated input

Parameters
MEBuilder (*coralme.builder.main.MEBuilder*)

Role: Reconstruct a ME-model from the information contained in class Organism.

This class was based almost entirely from the original ECOLIme code in build_me_model.py. Adaptations to this code were necessary to make it applicable to other organisms.

2.4.4 Homology()

class coralme.builder.homology.**Homology**(*org*, *ref*, *evalue=False*, *verbose=False*)

Homology class for storing information about homology of the main and reference organisms.

This class contains methods to predict and process homology of the main and reference organisms. Homology is inferred from the reciprocal best hits of a BLAST. The results are used to update and complement the attributes of the class Organism.

Parameters

- **org** (*str*) – Identifier of the main organism. Has to be the same as its containing folder name.
- **ref** (*str*) – Identifier of the reference organism. Has to be the same as its containing folder name.
- **evalue** (*float*) – E-value cutoff to call enzyme homologs from the BLAST. Two reciprocal best hits are considered homologs if their E-value is less than this parameter.

Role: Generate and store information about homology of the main and reference organisms.

This class contains methods to predict and process homology of the main and reference organisms. Homology is inferred from the reciprocal best hits of a BLAST. The results are used to update and complement the attributes of the class Organism.

2.4.5 Curation()

Role: Handle manual curation.

This class contains methods to handle the manual curation that is in building_data/

- **termination_subreactions.txt**
  Input here will define translation termination subreactions and their machinery.

class coralme.builder.curation.**TerminationSubreactions**(*org*, *id='termination_subreactions'*, *config={}*, *file='termination_subreactions.txt'*, *name='Translation termination subreactions'*)

Reads manual input to define translation termination subreactions.

This class creates the property "termination_subreactions" from the manual inputs in termination_subreactions.txt in an instance of Organism.

Input here will define translation termination subreactions and their machinery.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

termination_subreactions.txt :
subreaction enzymes PrfA_mono_mediated_termination PrfA_mono ...

- **peptide_release_factors.txt**
  Input here will define peptide release factors.

class coralme.builder.curation.**PeptideReleaseFactors**(*org*, *id='peptide_release_factors'*, *config={}*, *file='peptide_release_factors.txt'*, *name='Peptide release factors'*)

Reads manual input to define peptide release factors.

This class creates the property "peptide_release_factors" from the manual inputs in peptide_release_factors.txt in an instance of Organism.

Input here will define peptide release factors.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

peptide_release_factors.txt :
release_factor enzyme UAA generic_RF ...

- **rna_degradosome.txt**
  Input here will define the composition of the RNA degradosome.

class coralme.builder.curation.**RNADegradosome**(*org*, *id='rna_degradosome'*, *config={}*, *file='rna_degradosome.txt'*, *name='RNA degradosome composition'*)

Reads manual input to add RNA degradosome composition.

This class creates the property "rna_degradosome" from the manual inputs in rna_degradosome.txt in an instance of Organism.

Input here will define the composition of the RNA degradosome.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

rna_degradosome.txt :
enzymes Eno_dim_mod_mg2(4) ...

- **special_trna_subreactions.txt**
  Input here will define special tRNA subreactions, such as tRNA-Sec (selenocysteine) synthesis from tRNA-Ser.

class coralme.builder.curation.**SpecialtRNASubreactions**(*org*, *id='special_trna_subreactions'*, *config={}*, *file='special_trna_subreactions.txt'*, *name='Special tRNA subreactions'*)

Reads manual input to define special tRNA subreactions.

This class creates the property "special_trna_subreactions" from the manual inputs in special_trna_subreactions.txt in an instance of Organism.

Input here will define special tRNA subreactions, such as tRNA-Sec (selenocysteine) synthesis from tRNA-Ser.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

special_trna_subreactions.txt :
subreaction enzymes PrfA_mono_mediated_termination PrfA_mono ...

- **lipoprotein_precursors.txt**
  Input here will add lipoprotein precursors.

class coralme.builder.curation.**LipoproteinPrecursors**(*org*, *id='lipoprotein_precursors'*, *config={}*, *file='lipoprotein_precursors.txt'*, *name='Lipoprotein precursors'*)

Reads manual input to add lipoprotein precursors.

This class creates the property "lipoprotein_precursors" from the manual inputs in lipoprotein_precursors.txt in an instance of Organism.

Input here will add lipoprotein precursors.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

lipoprotein_precursors.txt :
name,gene AcrA,b0463 …

- **special_modifications.txt**
  Input here will define machinery for special modifications. These modifications are a set of pre-defined modifications that are used in ME-models.

class coralme.builder.curation.**SpecialModifications**(*org*, *id='special_modifications'*, *config={}*, *file='special_modifications.txt'*, *name='Special protein modifications'*)

Reads manual input to define machinery for special modifications.

This class creates the property "special_modifications" from the manual inputs in special_modifications.txt in an instance of Organism.

Input here will define machinery for special modifications. These modifications are a set of pre-defined modifications that are used in ME-models.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

special_modifications.txt :
modification enzymes stoich fes_transfer CPLX0-7617,IscA_tetra,CPLX0-7824 …

- **excision_machinery.txt**
  Input here will define machinery for excision.

class coralme.builder.curation.**ExcisionMachinery**(*org*, *id='excision_machinery'*, *config={}*, *file='excision_machinery.txt'*, *name='Excision machinery'*)

Reads manual input to define machinery for excision.

This class creates the property "excision_machinery" from the manual inputs in excision_machinery.txt in an instance of Organism.

Input here will define machinery for excision.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

excision_machinery.txt :
mechanism enzymes rRNA_containing RNase_E_tetra_mod_zn2(2), … …

- **orphan_and_spont_reactions.txt**
  Input here will mark reactions as orphan or spontaneous. Orphan reactions will be associated with CPLX_dummy, and spontaneous ones will not require enzymes for flux.

class coralme.builder.curation.**OrphanSpontReactions**(*org*, *id='orphan_and_spont_reactions'*, *config={}*, *file='orphan_and_spont_reactions.txt'*, *name='Orphan and spontaneous reactions'*)

Reads manual input to add reactions to the ME-model.

This class creates the property "orphan_and_spont_reactions" from the manual inputs in orphan_and_spont_reactions.txt in an instance of Organism.

Input here will mark reactions as orphan or spontaneous. Orphan reactions will be associated with CPLX_dummy, and spontaneous ones will not require enzymes for flux.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

orphan_and_spont_reactions.txt :
name description is_reversible is_spontaneous subsystems CODH_Fe_loading Loading of Fe false true …

- **enzyme_reaction_association.txt**
  Input here will create the association between enzymes and reactions in the ME-model.

class coralme.builder.curation.**EnzymeReactionAssociation**(*org*, *id='enz_rxn_assoc_df'*, *config={}*, *file='enzyme_reaction_association.txt'*, *name='Enzyme-reaction associations'*)

Reads manual input to specify enzyme-reaction associations.

This class creates the property "enz_rxn_assoc_df" from the manual inputs in enzyme_reaction_association.txt in an instance of Organism.

Input here will create the association between enzymes and reactions in the ME-model.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

enzyme_reaction_association.txt :
Reaction Complexes ADNt2pp NUPG-MONOMER OR NUPC-MONOMER …

- **peptide_compartment_and_pathways.txt**
  Input here will modify protein locations, and translocation pathways in the ME-model.

class coralme.builder.curation.**ProteinLocation**(*org*, *id='protein_location'*, *config={}*, *file='peptide_compartment_and_pathways.txt'*, *name='Protein location'*)

Reads manual input to add protein locations.

This class creates the property "protein_location" from the manual inputs in peptide_compartment_and_pathways.txt in an instance of Organism.

Input here will modify protein locations, and translocation pathways in the ME-model.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

peptide_compartment_and_pathways.txt :
Complex Complex_compartment Protein Protein_compartment translocase_pathway BSU02690-MONOMER Plasma_Membrane BSU02690() Plasma_Membrane s …

- **translocation_pathways.txt**
  Input here will define translocation pathways and their machinery.

class coralme.builder.curation.**TranslocationPathways**(*org*, *id='translocation_pathways'*, *config={}*, *file='translocation_pathways.txt'*, *name='Translocation pathways'*)

Reads manual input to define translocation pathways.

This class creates the property "translocation_pathways" from the manual inputs in translocation_pathways.txt in an instance of Organism.

Input here will define translocation pathways and their machinery.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

translocation_pathways.txt :
pathway enzyme sec BSU27650-MONOMER sec BSU35300-MONOMER sec secYEG …

- **rna_modification.txt**
  Input here will define enzymes that perform RNA modifications for either rRNA or tRNA in the ME-model.

class coralme.builder.curation.**RNAModificationMachinery**(*org*, *id='rna_modification_df'*, *config={}*, *file='rna_modification.txt'*, *name='RNA Modification machinery'*)

Reads manual input to add RNA modification machinery.

This class creates the property "rna_modification_df" from the manual inputs in rna_modification.txt in an instance of Organism.

Input here will define enzymes that perform RNA modifications for either rRNA or tRNA in the ME-model.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

rna_modification.txt :
modification positions type enzymes source D 16,17,20,20A,21 tRNA DusB_mono …

- **ribosomal_proteins.txt**
  Input here will define the composition of the ribosome.

class coralme.builder.curation.**RibosomeStoich**(*org*, *id='ribosome_stoich'*, *config={}*, *file='ribosomal_proteins.txt'*, *name='Ribosomal proteins'*)

Reads manual input to define ribosome composition.

This class creates the property "ribosome_stoich" from the manual inputs in ribosomal_proteins.txt in an instance of Organism.

Input here will define the composition of the ribosome.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

ribosomal_proteins.txt :
subunits proteins 30S RpsD_mono,… 50S generic_23s_rRNAs,generic_5s_rRNAs,RplA_mono,…

- **rho_independent.txt**
  Input here will mark genes with rho independent transcription termination.

class coralme.builder.curation.**RhoIndependent**(*org*, *id='rho_independent'*, *config={}*, *file='rho_independent.txt'*, *name='Genes with rho-independent termination'*)

Reads manual input to define genes with rho independent termination.

This class creates the property "rho_independent" from the manual inputs in rho_independent.txt in an instance of Organism.

Input here will mark genes with rho independent transcription termination.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

rho_independent.txt :
id b0344 …

- **sigma_factors.txt**
  Input here will mark proteins for N-terminal methionine cleavage in the ME-model.

class coralme.builder.curation.**Sigmas**(*org*, *id='sigmas'*, *config={}*, *file='sigma_factors.txt'*, *name='Sigma factors'*)

Reads manual input to modify or add sigma factors.

This class creates the property "sigmas" from the manual inputs in sigma_factors.txt in an instance of Organism.

Input here will mark proteins for N-terminal methionine cleavage in the ME-model.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

sigma_factors.txt :
sigma,complex,genes,name RpoH_mono,RNAP_32H,b3461,"RNA polymerase, sigma 32 (sigma H) factor" …

- **cleaved_methionine.txt**
  Input here will mark proteins for N-terminal methionine cleavage in the ME-model.

class coralme.builder.curation.**CleavedMethionine**(*org*, *id='cleaved_methionine'*, *config={}*, *file='cleaved_methionine.txt'*, *name='Proteins with N-terminal methionine cleavage'*)

Reads manual input to mark proteins that undergo N-terminal methionine cleavage.

This class creates the property "cleaved_methionine" from the manual inputs in cleaved_methionine.txt in an instance of Organism.

Input here will mark proteins for N-terminal methionine cleavage in the ME-model.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

cleaved_methionine.txt :
cleaved_methionine_genes b4154 …

- **folding_dict.txt**
  Input here will define folding pathways for proteins.

class coralme.builder.curation.**FoldingDict**(*org*, *id='folding_dict'*, *config={}*, *file='folding_dict.txt'*, *name='Protein to folding machinery associations'*)

Reads manual input to define folding pathways for proteins.

This class creates the property "folding_dict" from the manual inputs in folding_dict.txt in an instance of Organism.

Input here will define folding pathways for proteins.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

folding_dict.txt :
mechanism enzymes GroEL_dependent_folding b0014, … …

- **translocation_multipliers.txt**
  Input here will modify how many pores are required for the translocation of a protein.

class coralme.builder.curation.**TranslocationMultipliers**(*org*, *id='translocation_multipliers'*, *config={}*, *file='translocation_multipliers.txt'*, *name='Translocation multipliers'*)

Reads manual input to define translocation multipliers.

This class creates the property "translocation_multipliers" from the manual inputs in translocation_multipliers.txt in an instance of Organism.

Input here will modify how many pores are required for the translocation of a protein.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

translocation_multipliers.txt :
Gene,YidC_MONOMER,TatE_MONOMER,TatA_MONOMER b1855,2.0,0.0,0.0 …

- **subreaction_matrix.txt**
  Input here will define subreactions in the ME-model.

class coralme.builder.curation.**SubreactionMatrix**(*org*, *id='subreaction_matrix'*, *config={}*, *file='subreaction_matrix.txt'*, *name='Matrix of subreaction stoichiometries'*)

Reads manual input to add subreactions.

This class creates the property "subreaction_matrix" from the manual inputs in subreaction_matrix.txt in an instance of Organism.

Input here will define subreactions in the ME-model.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

subreaction_matrix.txt :
Reaction Metabolites Stoichiometry mod_acetyl_c accoa_c -1.0 mod_acetyl_c coa_c +1.0 …

- **me_metabolites.txt**
  Input here will mark metabolites in the M-model for replacement with their corrected E-matrix component.

class coralme.builder.curation.**MEMetabolites**(*org*, *id='me_mets'*, *config={}*, *file='me_metabolites.txt'*, *name='Metabolites to substitute from M-model'*)

Reads manual input to replace metabolites in the M-model.

This class creates the property "me_mets" from the manual inputs in me_metabolites.txt in an instance of Organism.

Input here will mark metabolites in the M-model for replacement with their corrected E-matrix component.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

me_metabolites.txt :
id me_id name formula compartment type sufbcd_c CPLX0-1341 SufBCD complex REPLACE ...

- **elongation_subreactions.txt**
  Input here will define translation elongation subreactions and their machinery.

class coralme.builder.curation.**ElongationSubreactions**(*org*, *id='elongation_subreactions'*, *config={}*, *file='elongation_subreactions.txt'*, *name='Translation elongation subreactions'*)

Reads manual input to define translation elongation subreactions.

This class creates the property "elongation_subreactions" from the manual inputs in elongation_subreactions.txt in an instance of Organism.

Input here will define translation elongation subreactions and their machinery.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

elongation_subreactions.txt :
subreaction enzymes FusA_mono_elongation FusA_mono ...

- **subsystem_classification.txt**
  Input here will classify subsystems in umbrella classifications which are then used to set a median Keff and correct it with the complex SASA.

class coralme.builder.curation.**SubsystemClassification**(*org*, *id='subsystem_classification'*, *config={}*, *file='subsystem_classification.txt'*, *name='Classification of subsystems'*)

Reads manual input to classify subsystems for Keff estimation.

This class creates the property "subsystem_classification" from the manual inputs in subsystem_classification.txt in an instance of Organism.

Input here will classify subsystems in umbrella classifications which are then used to set a median Keff and correct it with the complex SASA.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

subsystem_classification.txt : subsystem central_CE central_AFN intermediate secondary other S_Amino_acids_and_related_molecules 0 1 0 0 0 …

- **reaction_matrix.txt**
  Input here will define reactions directly in the ME-model. Definitions here will be added to the ME-model after processing the M-model into the ME-model.

class coralme.builder.curation.**ReactionMatrix**(*org*, *id='reaction_matrix'*, *config={}*, *file='reaction_matrix.txt'*, *name='Matrix of reaction stoichiometries'*)

Reads manual input to add reactions to the ME-model.

This class creates the property "reaction_matrix" from the manual inputs in reaction_matrix.txt in an instance of Organism.

Input here will define reactions directly in the ME-model. Definitions here will be added to the ME-model after processing the M-model into the ME-model.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

reaction_matrix.txt :
Reaction Metabolites Stoichiometry Cs_cyto_import cs_p -1.0 Cs_cyto_import h_c 1.0 Cs_cyto_import cs_c 1.0 Cs_cyto_import h_p -1.0 …

- **lipid_modifications.txt**
  Input here will define enzymes that perform lipid modifications.

class coralme.builder.curation.**LipidModifications**(*org*, *id='lipid_modifications'*, *config={}*, *file='lipid_modifications.txt'*, *name='Lipid modifications'*)

Reads manual input to define lipid modification machinery.

This class creates the property "lipid_modifications" from the manual inputs in lipid_modifications.txt in an instance of Organism.

Input here will define enzymes that perform lipid modifications.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

lipid_modifications.txt :
lipid_mod enzymes pg_pe_160 Lgt_MONOMER,LspA_MONOMER …

- **amino_acid_trna_synthetase.txt**
  Input here will define amino acid tRNA ligases.

class coralme.builder.curation.**AminoacidtRNASynthetase**(*org*, *id='amino_acid_trna_synthetase'*, *config={}*, *file='amino_acid_trna_synthetase.txt'*, *name='Amino acid to tRNA synthetase associations'*)

Reads manual input to define amino acid tRNA ligases.

This class creates the property "amino_acid_trna_synthetase" from the manual inputs in amino_acid_trna_synthetase.txt in an instance of Organism.

Input here will define amino acid tRNA ligases.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

amino_acid_trna_synthetase.txt :
amino_acid enzyme ala__L_c Ala_RS_tetra_mod_zn2(4) …

- **initiation_subreactions.txt**
  Input here will define translation initiation subreactions and their machinery.

class coralme.builder.curation.**InitiationSubreactions**(*org*, *id='initiation_subreactions'*, *config={}*, *file='initiation_subreactions.txt'*, *name='Translation initiation subreactions'*)

Reads manual input to define translation initiation subreactions.

This class creates the property "initiation_subreactions" from the manual inputs in initiation_subreactions.txt in an instance of Organism.

Input here will define translation initiation subreactions and their machinery.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

initiation_subreactions.txt :
subreaction enzymes Translation_initiation_factor_InfA InfA_mono …

- **post_transcriptional_modification_of_RNA.txt**
  Input here will define RNA genes that undergo modifications.

class coralme.builder.curation.**RNAModificationTargets**(*org*, *id='rna_modification_targets'*, *config={}*, *file='post_transcriptional_modification_of_RNA.txt'*, *name='RNA modification targets'*)

Reads manual input to add RNA modification targets.

This class creates the property "rna_modification_targets" from the manual inputs in post_transcriptional_modification_of_RNA.txt in an instance of Organism.

Input here will define RNA genes that undergo modifications.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

post_transcriptional_modification_of_RNA.txt :
bnum position modification b0202 20A D …

- **protein_corrections.txt**
  Input here will add, modify complexes in the ME-model, as well as add, modify their modifications. You can add a complex modification ID in the replace column, which will remove that modified complex and replace it with your manually added one.

class coralme.builder.curation.**ManualComplexes**(*org*, *id='manual_complexes'*, *config={}*, *file='protein_corrections.txt'*, *name='Protein corrections'*)

Reads manual input to modify or add complexes.

This class creates the property "manual_complexes" from the manual inputs in protein_corrections.txt in an instance of Organism.

Input here will add, modify complexes in the ME-model, as well as add, modify their modifications. You can add a complex modification ID in the replace column, which will remove that modified complex and replace it with your manually added one.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

This example adds SufBCD with their subunits and modifications, while removing SufBCD_mod_2fe3s(1).

protein_corrections.txt :
complex_id,name,genes,mod,replace SufBCD,SufBC2D Fe-S cluster scaffold complex,BSU32670(1) AND BSU32710(2) AND BSU32700(1),2fe2s(1),SufBCD_mod_2fe3s(1) …

property org_data

Final dataset stored in Organism instance

- **reaction_median_keffs.txt**
  Input here will define median Keffs for estimation of Keffs using the SASA method.
- **transcription_subreactions.txt**
  Input here will define machinery for transcription subreactions. These subreactions are a set of pre-defined subreactions that are used in ME-models.

class coralme.builder.curation.**TranscriptionSubreactions**(*org*, *id='transcription_subreactions'*, *config={}*, *file='transcription_subreactions.txt'*, *name='Transcription subreactions'*)

Reads manual input to define transcription subreactions.

This class creates the property "transcription_subreactions" from the manual inputs in transcription_subreactions.txt in an instance of Organism.

Input here will define machinery for transcription subreactions. These subreactions are a set of pre-defined subreactions that are used in ME-models.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

transcription_subreactions.txt :
mechanism enzymes Transcription_normal_rho_independent Mfd_mono_mod_mg2(1),NusA_mono,NusG_mono,GreA_mo
…

- **generic_dict.txt**
  Input here will define generics.

class coralme.builder.curation.**GenericDict**(*org*, *id='generic_dict'*, *config={}*, *file='generic_dict.txt'*, *name='Dictionary of generic complexes'*)

Reads manual input to define generics.

This class creates the property "generic_dict" from the manual inputs in generic_dict.txt in an instance of Organism.

Input here will define generics.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

generic_dict.txt :
generic_component enzymes generic_16Sm4Cm1402 RsmH_mono,RsmI_mono …

- **ribosome_subreactions.txt**
  Input here will define enzymes that perform a ribosome subreaction.

class coralme.builder.curation.**RibosomeSubreactions**(*org*, *id='ribosome_subreactions'*, *config={}*, *file='ribosome_subreactions.txt'*, *name='Ribosomal subreactions'*)

Reads manual input to define ribosome subreactions.

This class creates the property "ribosome_subreactions" from the manual inputs in ribosome_subreactions.txt in an instance of Organism.

Input here will define enzymes that perform a ribosome subreaction.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

ribosome_subreactions.txt :
subreaction enzyme gtp_bound_30S_assembly_factor_phase1 BSU16650-MONOMER …

- **reaction_corrections.txt**
  Input here will modify reactions at the M-model stage before ME-model building.

class coralme.builder.curation.**ReactionCorrections**(*org*, *id='reaction_corrections'*, *config={}*, *file='reaction_corrections.txt'*, *name='Reaction corrections'*)

Reads manual input to modify reactions in the M-model.

This class creates the property "reaction_corrections" from the manual inputs in reaction_corrections.txt in an instance of Organism.

Input here will modify reactions at the M-model stage before ME-model building.

Parameters
org (*coralme.builder.organism.Organism*) – Organism object.

Examples

reaction_corrections.txt :
reaction_id,name,gene_reaction_rule,reaction,notes COBALT2tpp,cobalt transport in via permease (no H+),BSU24740,cobalt2_e –> cobalt2_c,No notes …

- **TUs_from_biocyc.txt**
  Input here will modify transcriptional unit information.

[ ]:

2.5 Reconstructing with BioCyc information and curated files

In this tutorial we will reconstruct a *Bacillus subtilis* ME-model from just the input files.

2.5.1 Import libraries

[1]:
```
from coralme.builder.main import MEBuilder
import coralme
```

2.5.2 Path to configuration files

For more information about these files see *Description of inputs*. The configuration and layout used here is as shown in this B. subtilis ME-model repository.

[2]:
```
organism = './bsubtilis/organism.json'
inputs = './bsubtilis/input.json'
```

2.5.3 Create MEBuilder instance

For more information about this class see *Architecture of coralME*

[3]:
```
builder = MEBuilder(*[organism, inputs])
```

2.5.4 Generate files

This corresponds to *Synchronyze* and *Complement* steps in *Architecture of coralME*

[4]:
```
builder.generate_files(overwrite=True)
```

```
Initiating file processing...
~ Processing files for bsubtilis...
Set parameter Username
Academic license - for non-commercial use only - expires 2025-09-03
Checking M-model metabolites...                                    : 100.0%|| 990/  990 [00:00<00:00]
Checking M-model genes...                                          : 100.0%|| 844/  844 [00:00<00:00]
Checking M-model reactions...                                      : 100.0%|| 1250/ 1250 [00:00<00:00]
Generating complexes dataframe from optional proteins file...      : 100.0%|| 4554/ 4554 [00:00<00:00]
Syncing optional genes file...                                     : 100.0%|| 4541/ 4541 [00:00<00:00]
Looking for duplicates within datasets...                          : 100.0%||  5/    5 [00:00<00:00]
Gathering ID occurrences across datasets...                        : 100.0%||10647/10647 [00:00<00:00]
Solving duplicates across datasets...                              : 0.0%|    |     0/    0 [00:00<?]
```

(continues on next page)

(continued from previous page)

```
Pruning GenBank...                                                    : 100.0%||   ␣
↪ 1/    1 [00:01<00:00]
Updating Genbank file with optional files...                          : 100.0%|| ␣
↪4541/ 4541 [00:00<00:00]
Syncing optional files with genbank contigs...                        : 100.0%||   ␣
↪ 6/    6 [00:04<00:00]
Modifying metabolites with manual curation...                         : 0.0%|      ␣
↪    |     0/    0 [00:00<?]
Modifying metabolic reactions with manual curation...                 : 100.0%||   ␣
↪36/   36 [00:00<00:00]
```

```
unknown metabolite '4crsol_c' created
unknown metabolite 'dad__5_c' created
unknown metabolite 'dhgly_c' created
unknown metabolite '5drib_c' created
unknown metabolite 'cu_e' created
unknown metabolite 'cu_c' created
unknown metabolite 'cbl1_e' created
unknown metabolite 'cbl1_c' created
```

```
Adding manual curation of complexes...                                : 100.0%|| ␣
↪438/  438 [00:00<00:00]
Getting sigma factors...                                              : 100.0%||   ␣
↪38/   38 [00:00<00:00]
Getting generics from Genbank contigs...                              : 100.0%||   ␣
↪ 6/    6 [00:00<00:00]
Getting TU-gene associations from optional TUs file...                : 100.0%|| ␣
↪1647/ 1647 [00:00<00:00]
Adding protein location...                                            : 100.0%|| ␣
↪4683/ 4683 [00:00<00:00]
Purging M-model genes...                                              : 100.0%||  ␣
↪851/  851 [00:00<00:00]
Getting enzyme-reaction associations...                               : 100.0%|| ␣
↪1262/ 1262 [00:00<00:00]
```

```
Reading bsubtilis done.
```

```
Gathering M-model compartments...                                     : 100.0%||   ␣
↪ 2/    2 [00:00<00:00]
Fixing compartments in M-model metabolites...                         : 100.0%||  ␣
↪998/  998 [00:00<00:00]
Fixing missing names in M-model reactions...                          : 100.0%|| ␣
↪1262/ 1262 [00:00<00:00]
```

```
~ Processing files for iJL1678b...
```

```
Checking M-model metabolites...                                       : 100.0%|| ␣
↪1660/ 1660 [00:00<00:00]
Checking M-model genes...                                             : 100.0%|| ␣
↪1271/ 1271 [00:00<00:00]
Checking M-model reactions...                                         : 100.0%|| ␣
↪2377/ 2377 [00:00<00:00]
Looking for duplicates within datasets...                             : 100.0%||   ␣
↪ 5/    5 [00:00<00:00]
Gathering ID occurrences across datasets...                           : 100.0%|| ␣
```

(continues on next page)

(continued from previous page)

```
↪8517/ 8517 [00:00<00:00]
Solving duplicates across datasets...                                    : 0.0%|      ␣
↪   |     0/    0 [00:00<?]
Getting sigma factors...                                                 : 100.0%||   ␣
↪ 7/    7 [00:00<00:00]
Getting TU-gene associations from optional TUs file...                   : 100.0%|| ␣
↪1647/ 1647 [00:00<00:00]
Adding protein location...                                               : 100.0%|| ␣
↪1304/ 1304 [00:00<00:00]
Purging M-model genes...                                                 : 100.0%|| ␣
↪1271/ 1271 [00:00<00:00]
```

```
Reading iJL1678b done.
~ Running BLAST with 1 threads...
```

```
Converting Genbank contigs to FASTA for BLAST...                         : 100.0%||   ␣
↪ 6/    6 [00:00<00:00]
Converting Genbank contigs to FASTA for BLAST...                         : 100.0%||   ␣
↪ 1/    1 [00:00<00:00]
```

```
BLAST done.
```

```
Updating translocation machinery from homology...                        : 100.0%||   ␣
↪ 9/    9 [00:00<00:00]
Updating protein location from homology...                               : 100.0%||  ␣
↪514/  514 [00:00<00:00]
Updating translocation multipliers from homology...                      : 100.0%||   ␣
↪ 3/    3 [00:00<00:00]
Updating lipoprotein precursors from homology...                         : 100.0%||   ␣
↪14/   14 [00:00<00:00]
Updating cleaved-methionine proteins from homology...                    : 100.0%||  ␣
↪343/  343 [00:00<00:00]
Mapping M-metabolites to E-metabolites...                                : 100.0%||  ␣
↪147/  147 [00:00<00:00]
Updating generics from homology...                                       : 100.0%||   ␣
↪10/   10 [00:00<00:00]
Updating folding from homology...                                        : 100.0%||   ␣
↪ 2/    2 [00:00<00:00]
Updating ribosome subreaction machinery from homology...                 : 100.0%||   ␣
↪ 3/    3 [00:00<00:00]
Updating tRNA synthetases from homology...                               : 100.0%||   ␣
↪20/   20 [00:00<00:00]
Updating peptide release factors from homology...                        : 100.0%||   ␣
↪ 3/    3 [00:00<00:00]
Updating transcription subreactions machinery from homology...           : 100.0%||   ␣
↪ 4/    4 [00:00<00:00]
Updating translation initiation subreactions from homology...            : 100.0%||   ␣
↪ 4/    4 [00:00<00:00]
Updating translation elongation subreactions from homology...            : 100.0%||   ␣
↪ 2/    2 [00:00<00:00]
Updating translation termination subreactions from homology...           : 100.0%||   ␣
↪ 9/    9 [00:00<00:00]
Updating special tRNA subreactions from homology...                      : 100.0%||   ␣
↪ 1/    1 [00:00<00:00]
```

(continues on next page)

(continued from previous page)

```
Updating RNA degradosome composition from homology...                    : 100.0%||  ␣
↪ 1/    1 [00:00<00:00]
Updating excision machinery from homology...                             : 100.0%||  ␣
↪ 3/    3 [00:00<00:00]
Updating tRNA subreactions from homology...                              : 100.0%||  ␣
↪ 6/    6 [00:00<00:00]
Updating lipid modification machinery from homology...                   : 100.0%||  ␣
↪ 2/    2 [00:00<00:00]
Fixing M-model metabolites with homology...                              : 100.0%||  ␣
↪998/  998 [00:00<00:00]
Updating enzyme reaction association...                                  : 100.0%||  ␣
↪922/  922 [00:00<00:00]
Getting tRNA to codon dictionary from NC_000964.3                        : 100.0%||  ␣
↪4537/ 4537 [00:03<00:00]
Getting tRNA to codon dictionary from BSU_misc_RNA_38                    : 100.0%||  ␣
↪ 2/    2 [00:00<00:00]
Getting tRNA to codon dictionary from G8J2-183                           : 100.0%||  ␣
↪ 2/    2 [00:00<00:00]
Getting tRNA to codon dictionary from G8J2-180                           : 100.0%||  ␣
↪ 2/    2 [00:00<00:00]
Getting tRNA to codon dictionary from G8J2-181                           : 100.0%||  ␣
↪ 2/    2 [00:00<00:00]
Getting tRNA to codon dictionary from G8J2-182                           : 100.0%||  ␣
↪ 2/    2 [00:00<00:00]
Checking defined translocation pathways...                               : 100.0%||  ␣
↪ 9/    9 [00:00<00:00]
Getting reaction Keffs...                                                : 100.0%||  ␣
↪922/  922 [00:00<00:00]
```

```
Writting the Organism-Specific Matrix...
Organism-Specific Matrix saved to ./bsubtilis/building_data/automated-org-with-refs.xlsx␣
↪file.
File processing done.
```

2.5.5 Build ME-model

This corresponds to *Build* in *Architecture of coralME*

[5]:
```
builder.build_me_model(overwrite=False)
```

```
Initiating ME-model reconstruction...
```

```
Adding biomass constraint(s) into the ME-model...                        : 100.0%||  ␣
↪11/   11 [00:00<00:00]
```

```
Read LP format model from file /tmp/tmp7trh8uhg.lp
Reading time = 0.00 seconds
: 997 rows, 2524 columns, 10562 nonzeros
```

```

```

```
Read LP format model from file /tmp/tmp575tct7k.lp
Reading time = 0.00 seconds
: 997 rows, 2520 columns, 10544 nonzeros
```

```
Adding Metabolites from M-model into the ME-model...                    : 100.0%|| ␣
↪1087/ 1087 [00:00<00:00]
Adding Reactions from M-model into the ME-model...                      : 100.0%|| ␣
↪1277/ 1277 [00:00<00:00]
Adding Transcriptional Units into the ME-model from user input...       : 100.0%|| ␣
↪1515/ 1515 [00:09<00:00]
Adding features from contig NC_000964.3 into the ME-model...            : 100.0%|| ␣
↪4537/ 4537 [00:09<00:00]
Adding features from contig BSU_misc_RNA_38 into the ME-model...        : 100.0%||   ␣
↪ 2/    2 [00:00<00:00]
Adding features from contig G8J2-183 into the ME-model...               : 100.0%||   ␣
↪ 2/    2 [00:00<00:00]
Adding features from contig G8J2-180 into the ME-model...               : 100.0%||   ␣
↪ 2/    2 [00:00<00:00]
Adding features from contig G8J2-181 into the ME-model...               : 100.0%||   ␣
↪ 2/    2 [00:00<00:00]
Adding features from contig G8J2-182 into the ME-model...               : 100.0%||   ␣
↪ 2/    2 [00:00<00:00]
Updating all TranslationReaction and TranscriptionReaction...           : 100.0%|| ␣
↪8389/ 8389 [00:21<00:00]
Removing SubReactions from ComplexData...                               : 100.0%|| ␣
↪5007/ 5007 [00:00<00:00]
Adding ComplexFormation into the ME-model...                            : 100.0%|| ␣
↪5007/ 5007 [00:00<00:00]
Adding Generic(s) into the ME-model...                                  : 100.0%||   ␣
↪10/   10 [00:00<00:00]
Processing StoichiometricData in ME-model...                            : 100.0%|| ␣
↪1045/ 1045 [00:00<00:00]
```

```
ME-model was saved in the ./bsubtilis/ directory as MEModel-step1-bsubtilis.pkl
```

```
Adding tRNA synthetase(s) information into the ME-model...              : 100.0%||  ␣
↪306/  306 [00:00<00:00]
Adding tRNA modification SubReactions...                                : 100.0%||   ␣
↪19/   19 [00:00<00:00]
Associating tRNA modification enzyme(s) to tRNA(s)...                   : 100.0%||   ␣
↪33/   33 [00:00<00:00]
Adding SubReactions into TranslationReactions...                        : 100.0%|| ␣
↪4328/ 4328 [00:01<00:00]
Adding RNA Polymerase(s) into the ME-model...                           : 100.0%||   ␣
↪39/   39 [00:00<00:00]
Associating a RNA Polymerase to each Transcriptional Unit...            : 100.0%|| ␣
↪1515/ 1515 [00:00<00:00]
Processing ComplexData in ME-model...                                   : 100.0%||  ␣
↪322/  322 [00:00<00:00]
Adding ComplexFormation into the ME-model...                            : 100.0%|| ␣
↪5054/ 5054 [00:00<00:00]
Adding SubReactions into TranslationReactions...                        : 100.0%|| ␣
↪4328/ 4328 [00:08<00:00]
Adding Transcription SubReactions...                                    : 100.0%|| ␣
↪3513/ 3513 [00:00<00:00]
Processing StoichiometricData in SubReactionData...                     : 100.0%|| ␣
↪1650/ 1650 [00:00<00:00]
Adding reaction subsystems from M-model into the ME-model...            : 100.0%|| ␣
```

(continues on next page)

(continued from previous page)

```
↪1277/ 1277 [00:00<00:00]
Processing StoichiometricData in ME-model...                                : 100.0%|| ␣
↪1045/ 1045 [00:00<00:00]
Updating ME-model Reactions...                                              : 100.0%||␣
↪17480/17480 [01:25<00:00]
Updating all FormationReactions...                                          : 100.0%|| ␣
↪5054/ 5054 [00:00<00:00]
Recalculation of the elemental contribution in SubReactions...              : 100.0%|| ␣
↪277/  277 [00:00<00:00]
Updating all FormationReactions...                                          : 100.0%|| ␣
↪5054/ 5054 [00:00<00:00]
Updating FormationReactions involving a lipoyl prosthetic group...          : 100.0%|| ␣
↪ 3/    3 [00:00<00:00]
Updating FormationReactions involving a glycyl radical...                   : 0.0%|    ␣
↪   |     0/    0 [00:00<?]
Estimating effective turnover rates for reactions using the SASA method... : 100.0%|| ␣
↪2882/ 2882 [00:00<00:00]
Mapping effective turnover rates from user input...                         : 0.0%|    ␣
↪   |     0/    0 [00:00<?]
Setting the effective turnover rates using user input...                    : 100.0%|| ␣
↪2632/ 2632 [00:00<00:00]
Pruning unnecessary ComplexData reactions...                                : 100.0%|| ␣
↪5054/ 5054 [00:06<00:00]
Pruning unnecessary FoldedProtein reactions...                              : 0.0%|    ␣
↪   |     0/    0 [00:00<?]
Pruning unnecessary ProcessedProtein reactions...                           : 100.0%|| ␣
↪5647/ 5647 [00:01<00:00]
Pruning unnecessary TranslatedGene reactions...                             : 100.0%|| ␣
↪4640/ 4640 [00:09<00:00]
Pruning unnecessary TranscribedGene reactions...                            : 100.0%|| ␣
↪4540/ 4540 [00:05<00:00]
Pruning unnecessary Transcriptional Units...                                : 100.0%|| ␣
↪3513/ 3513 [00:03<00:00]
Pruning unnecessary ComplexData reactions...                                : 100.0%|| ␣
↪988/  988 [00:00<00:00]
Pruning unnecessary FoldedProtein reactions...                              : 0.0%|    ␣
↪   |     0/    0 [00:00<?]
Pruning unnecessary ProcessedProtein reactions...                           : 100.0%|| ␣
↪1354/ 1354 [00:00<00:00]
Pruning unnecessary TranslatedGene reactions...                             : 100.0%|| ␣
↪1354/ 1354 [00:00<00:00]
Pruning unnecessary TranscribedGene reactions...                            : 100.0%|| ␣
↪1159/ 1159 [00:02<00:00]
Pruning unnecessary Transcriptional Units...                                : 100.0%|| ␣
↪792/  792 [00:01<00:00]
Pruning unnecessary ComplexData reactions...                                : 100.0%|| ␣
↪965/  965 [00:00<00:00]
Pruning unnecessary FoldedProtein reactions...                              : 0.0%|    ␣
↪   |     0/    0 [00:00<?]
Pruning unnecessary ProcessedProtein reactions...                           : 100.0%|| ␣
↪1352/ 1352 [00:00<00:00]
Pruning unnecessary TranslatedGene reactions...                             : 100.0%|| ␣
```

(continues on next page)

(continued from previous page)

```
↪1352/ 1352 [00:00<00:00]
Pruning unnecessary TranscribedGene reactions...                    : 100.0%|| ␣
↪1157/ 1157 [00:01<00:00]
Pruning unnecessary Transcriptional Units...                        : 100.0%|| ␣
↪790/  790 [00:01<00:00]
Pruning unnecessary ComplexData reactions...                        : 100.0%|| ␣
↪964/  964 [00:00<00:00]
Pruning unnecessary FoldedProtein reactions...                      : 0.0%|    ␣
↪   |     0/    0 [00:00<?]
Pruning unnecessary ProcessedProtein reactions...                   : 100.0%|| ␣
↪1351/ 1351 [00:00<00:00]
Pruning unnecessary TranslatedGene reactions...                     : 100.0%|| ␣
↪1351/ 1351 [00:00<00:00]
Pruning unnecessary TranscribedGene reactions...                    : 100.0%|| ␣
↪1156/ 1156 [00:02<00:00]
Pruning unnecessary Transcriptional Units...                        : 100.0%|| ␣
↪789/  789 [00:00<00:00]
```

```
ME-model was saved in the ./bsubtilis/ directory as MEModel-step2-bsubtilis.pkl
ME-model reconstruction is done.
Number of metabolites in the ME-model is 4630 (+364.39%, from 997)
Number of reactions in the ME-model is 7758 (+514.74%, from 1262)
Number of genes in the ME-model is 1154 (+36.73%, from 844)
Number of missing genes from reconstruction with homology, but no function is 104. Check␣
↪the curation notes for more details.
```

2.5.6 Troubleshoot ME-model

This corresponds to *Find gaps* in *Architecture of coralME*

[6]:
```
builder.troubleshoot(growth_key_and_value = { builder.me_model.mu : 0.001 },solver=
↪"qminos")
```

```
The MINOS and quad MINOS solvers are a courtesy of Prof Michael A. Saunders. Please cite␣
↪Ma, D., Yang, L., Fleming, R. et al. Reliable and efficient solution of genome-scale␣
↪models of Metabolism and macromolecular Expression. Sci Rep 7, 40863 (2017). https://
↪doi.org/10.1038/srep40863

~ Troubleshooting started...
  Checking if the ME-model can simulate growth without gapfilling reactions...
  Original ME-model is feasible with a tested growth rate of 0.001000 1/h
~ Final step. Fully optimizing with precision 1e-6 and save solution into the ME-model...
  Gapfilled ME-model is feasible with growth rate 0.101279 (M-model: 0.117966).
ME-model was saved in the ./bsubtilis/ directory as MEModel-step3-bsubtilis-TS.pkl
```

Note: We set 0.001 as a standard value for feasibility checking, but feel free to modify it! Sometimes too high a value could put a significant strain on the model and give too many gaps to start with. Too low a value might not show you all the gaps needed.

2.5.7 Save as a JSON

[7]:
```
coralme.io.json.save_json_me_model(builder.me_model,"./bsubtilis/MEModel-step3-bsubtilis-TS.json")
```

```
2024-11-26 17:43:32,509 The metabolite 'pg160_p' must exist in the ME-model to calculate the element contribution.
2024-11-26 17:43:32,510 The metabolite '2agpg160_p' must exist in the ME-model to calculate the element contribution.
2024-11-26 17:43:32,510 The metabolite 'pe160_p' must exist in the ME-model to calculate the element contribution.
2024-11-26 17:43:32,510 The metabolite '2agpe160_p' must exist in the ME-model to calculate the element contribution.
2024-11-26 17:43:32,909 Metabolite '4fe4s_c' does not have a formula. If it is a 'Complex', its formula will be determined from amino acid composition and prosthetic groups stoichiometry. Otherwise, please add it to the M-model.
2024-11-26 17:43:32,910 Metabolite '4fe4s_c' does not have a formula. If it is a 'Complex', its formula will be determined from amino acid composition and prosthetic groups stoichiometry. Otherwise, please add it to the M-model.
2024-11-26 17:43:32,910 Metabolite '2fe2s_c' does not have a formula. If it is a 'Complex', its formula will be determined from amino acid composition and prosthetic groups stoichiometry. Otherwise, please add it to the M-model.
2024-11-26 17:43:32,910 Metabolite '2fe2s_c' does not have a formula. If it is a 'Complex', its formula will be determined from amino acid composition and prosthetic groups stoichiometry. Otherwise, please add it to the M-model.
2024-11-26 17:43:32,910 Metabolite 'hemed_c' does not have a formula. If it is a 'Complex', its formula will be determined from amino acid composition and prosthetic groups stoichiometry. Otherwise, please add it to the M-model.
2024-11-26 17:43:32,911 Metabolite 'hemed_c' does not have a formula. If it is a 'Complex', its formula will be determined from amino acid composition and prosthetic groups stoichiometry. Otherwise, please add it to the M-model.
2024-11-26 17:43:32,911 Metabolite 'fmnh2_c' does not have a formula. If it is a 'Complex', its formula will be determined from amino acid composition and prosthetic groups stoichiometry. Otherwise, please add it to the M-model.
2024-11-26 17:43:32,911 Metabolite 'fmnh2_c' does not have a formula. If it is a 'Complex', its formula will be determined from amino acid composition and prosthetic groups stoichiometry. Otherwise, please add it to the M-model.
2024-11-26 17:43:32,912 Metabolite 'tl_c' does not have a formula. If it is a 'Complex', its formula will be determined from amino acid composition and prosthetic groups stoichiometry. Otherwise, please add it to the M-model.
2024-11-26 17:43:32,912 Metabolite 'tl_c' does not have a formula. If it is a 'Complex', its formula will be determined from amino acid composition and prosthetic groups stoichiometry. Otherwise, please add it to the M-model.
2024-11-26 17:43:32,912 Metabolite 'cs_c' does not have a formula. If it is a 'Complex', its formula will be determined from amino acid composition and prosthetic groups stoichiometry. Otherwise, please add it to the M-model.
```

(continues on next page)

(continued from previous page)

```
2024-11-26 17:43:32,912 Metabolite 'cs_c' does not have a formula. If it is a 'Complex',␣
↪its formula will be determined from amino acid composition and prosthetic groups␣
↪stoichiometry. Otherwise, please add it to the M-model.
2024-11-26 17:43:32,913 The metabolite 'adocbl_c' must exist in the ME-model to␣
↪calculate the element contribution.
2024-11-26 17:43:32,913 The metabolite 'adocbl_c' must exist in the ME-model to␣
↪calculate the element contribution.
2024-11-26 17:43:32,913 The metabolite 'coo_c' must exist in the ME-model to calculate␣
↪the element contribution.
2024-11-26 17:43:32,913 The metabolite 'coo_c' must exist in the ME-model to calculate␣
↪the element contribution.
2024-11-26 17:43:32,913 The metabolite 'cosh_c' must exist in the ME-model to calculate␣
↪the element contribution.
2024-11-26 17:43:32,914 The metabolite 'cosh_c' must exist in the ME-model to calculate␣
↪the element contribution.
2024-11-26 17:43:32,914 The metabolite 'lipo_c' must exist in the ME-model to calculate␣
↪the element contribution.
2024-11-26 17:43:32,914 The metabolite 'lipo_c' must exist in the ME-model to calculate␣
↪the element contribution.
2024-11-26 17:43:32,914 Metabolite 'lipoyl_c' does not have a formula. If it is a
↪'Complex', its formula will be determined from amino acid composition and prosthetic␣
↪groups stoichiometry. Otherwise, please add it to the M-model.
2024-11-26 17:43:32,914 Metabolite 'lipoyl_c' does not have a formula. If it is a
↪'Complex', its formula will be determined from amino acid composition and prosthetic␣
↪groups stoichiometry. Otherwise, please add it to the M-model.
2024-11-26 17:43:32,915 The metabolite 'li_c' must exist in the ME-model to calculate␣
↪the element contribution.
2024-11-26 17:43:32,915 The metabolite 'li_c' must exist in the ME-model to calculate␣
↪the element contribution.
2024-11-26 17:43:32,915 The metabolite 'cl_c' must exist in the ME-model to calculate␣
↪the element contribution.
2024-11-26 17:43:32,915 The metabolite 'cl_c' must exist in the ME-model to calculate␣
↪the element contribution.
2024-11-26 17:43:32,915 The metabolite 'bmocogdp_c' must exist in the ME-model to␣
↪calculate the element contribution.
2024-11-26 17:43:32,915 The metabolite 'bmocogdp_c' must exist in the ME-model to␣
↪calculate the element contribution.
```

2.6 Reconstructing a ME-model from the OSM

The organism-specific matrix is a XLSX structure that contains all the mappings necessary to build a ME-model (see *Architecture of coralME*).

In this tutorial we will reconstruct a *Bacillus subtilis* ME-model from the OSM. This is useful once you have already run builder.generate_files() which performs the *Synchronize* and *Complement* steps (see *Architecture of coralME*).

This way is quicker but make sure not to overwrite your OSM re-running the reconstruction as in Tutorial 1, because all you manual curation will be lost.

2.6.1 Import libraries

```
[ ]: from IPython.display import display, HTML, Math, Markdown
display(HTML("<style>.container { width:95% !important; }</style>"))

from coralme.builder.main import MEBuilder
```

2.6.2 Path to configuration files

```
[ ]: organism = './bsubtilis/organism.json'
inputs = './bsubtilis/input.json'
```

2.6.3 Create MEBuilder instance

This time you create it from the yaml configuration file which is already generated and filled. This file condenses the configuration in one place, see an example here.

```
[ ]: builder = MEBuilder(*['./bsubtilis/coralme-config.yaml'])
```

2.6.4 Build ME-model

This corresponds to *Build* in *Architecture of coralME*

```
[ ]: builder.build_me_model(overwrite=False)
```

2.6.5 Troubleshoot ME-model

This corresponds to *Find gaps* in *Architecture of coralME*

```
[ ]: builder.troubleshoot(growth_key_and_value = { builder.me_model.mu : 0.001 }, solver=
↪"qminos")
```

Note: We set 0.001 as a standard value for feasibility checking, but feel free to modify it! Sometimes too high a value could put a significant strain on the model and give too many gaps to start with. Too low a value might not show you all the gaps needed.

```
[ ]: import coralme
coralme.io.json.save_json_me_model(builder.me_model,"./bsubtilis/MEModel-step3-bsubtilis-
↪TS.json")
```

2.7 Loading and inspecting ME-models

In this tutorial we will load and inspect the reconstructed *Bacillus subtilis* ME-model.

2.7.1 Import libraries

[1]:
```
from coralme.builder.main import MEBuilder
import coralme
```

2.7.2 Load as a JSON

Load the ME-model coming out of the Troubleshooter

[2]:
```
me = coralme.io.json.load_json_me_model("./bsubtilis/MEModel-step3-bsubtilis-TS.json")
```

```
Adding Metabolites into the ME-model...                                    : 100.0%|| ␣
↪4630/ 4630 [00:00<00:00]
Adding ProcessData into the ME-model...                                    : 100.0%|| ␣
↪4752/ 4752 [00:00<00:00]
Adding Reactions into the ME-model...                                      : 100.0%|| ␣
↪7758/ 7758 [00:14<00:00]
Updating ME-model Reactions...                                             : 100.0%|| ␣
↪6369/ 6369 [00:21<00:00]
```

2.7.3 Load as a pickle

[ ]:
```
# me = coralme.io.pickle.load_pickle_me_model("./bsubtilis/MEModel-step3-bsubtilis-TS.pkl
↪")
```

2.7.4 Inspecting ME-model properties

Summary of the ME-model

[3]:
```
me
```

[3]:
```
<MEModel coralME at 0x7f1c5c495b10>
```

[4]:
```
print("ME-model has {} reactions".format(len(me.reactions)))
```

```
ME-model has 7758 reactions
```

[5]:
```
print("ME-model has {} metabolites".format(len(me.metabolites)))
```

```
ME-model has 4630 metabolites
```

[6]:
```
print("ME-model has {} genes".format(len(me.all_genes)))
```

```
ME-model has 1154 genes
```

Reactions

Metabolic reaction

[7]:
```
r = next(i for i in me.reactions if isinstance(i,coralme.core.reaction.
↪MetabolicReaction))
r
```

[7]:
```
<MetabolicReaction 23CN2P1_REV_BSU07840-MONOMER at 0x7f1b98d6c9d0>
```

Translation reaction

[8]:
```
r = next(i for i in me.reactions if isinstance(i,coralme.core.reaction.
↪TranslationReaction))
r
```

[8]:
```
<TranslationReaction translation_BSU00090 at 0x7f1b9abcb310>
```

Transcription reaction

[9]:
```
r = next(i for i in me.reactions if isinstance(i,coralme.core.reaction.
↪TranscriptionReaction))
r
```

[9]:
```
<TranscriptionReaction transcription_TU8J2-1243_from_BSU25200-MONOMER at 0x7f1b9aea4700>
```

Formation reaction

[10]:
```
r = next(i for i in me.reactions if isinstance(i,coralme.core.reaction.ComplexFormation))
r
```

[10]:
```
<ComplexFormation formation_BSU00090-MONOMER at 0x7f1b98def070>
```

tRNA Charging reaction

[11]:
```
r = next(i for i in me.reactions if isinstance(i,coralme.core.reaction.
↪tRNAChargingReaction))
r
```

[11]:
```
<tRNAChargingReaction charging_tRNA_BSU_tRNA_5_AUU at 0x7f1b9918bbe0>
```

Post-translational modification reaction

[12]:
```
r = next(i for i in me.reactions if isinstance(i,coralme.core.reaction.
↪PostTranslationReaction))
r
```

[12]:
```
<PostTranslationReaction translocation_BSU07840_Cell_Wall at 0x7f1b98956650>
```

Mapped gene functions

[13]:
```
from collections import defaultdict
import pandas
d = defaultdict(int)
for g in me.all_genes:
    for f in g.functions:
        d[f] += 1
pandas.DataFrame.from_dict({"count":d})
```

[13]:

	count
Translation	209
tRNA-Charging	135
Metabolic:S_Nucleotides_and_nucleic_acids	67
Metabolic:S_Coenzymes_and_prosthetic_groups	76
Metabolic:S_Amino_acids_and_related_molecules	155
Metabolic:S_Other_function	16
Transcription	46
Metabolic:S_Carbohydrates_and_related_molecules	148
Metabolic:S_Cell_wall	40
Metabolic:S_Lipids	55
Post-translation	13
Metabolic:S_Transport	235
Metabolic:Not Determined	14
Metabolic:S_Membrane_bioenergetics	47
Metabolic: No subsystem	8
Metabolic:S_Phosphate_and_sulfur	7

Inspecting functions of a gene

[14]:
```
g = me.all_genes.get_by_id("RNA_BSU37160")
g.functions
```

[14]:
```
{'Transcription'}
```

Complexes formed by a gene

[15]:
```
g = me.all_genes.get_by_id("RNA_BSU37160")
g.complexes
```

[15]:
```
{<Complex CPLX8J2-30_mod_zn2(1)_mod_mg2(2) at 0x7f1b9b1d9db0>,
 <RNAP RNAP_BSU00980MONOMER at 0x7f1b9b1da530>,
 <RNAP RNAP_BSU01730MONOMER at 0x7f1b9b1da5c0>,
 <RNAP RNAP_BSU04730MONOMER at 0x7f1b9b1da5f0>,
 <RNAP RNAP_BSU09520MONOMER at 0x7f1b9b1da4a0>,
 <RNAP RNAP_BSU13450MONOMER at 0x7f1b9b1da410>,
 <RNAP RNAP_BSU14730MONOMER at 0x7f1b9b1da4d0>,
 <RNAP RNAP_BSU15320MONOMER at 0x7f1b9b1da680>,
 <RNAP RNAP_BSU15330MONOMER at 0x7f1b9b1da560>,
 <RNAP RNAP_BSU16470MONOMER at 0x7f1b9b1da650>,
 <RNAP RNAP_BSU23100MONOMER at 0x7f1b9b1da500>,
 <RNAP RNAP_BSU23450MONOMER at 0x7f1b9b1da440>,
 <RNAP RNAP_BSU25200MONOMER at 0x7f1b9b1da620>,
 <RNAP RNAP_BSU27120MONOMER at 0x7f1b9b1da470>,
 <RNAP RNAP_BSU34200MONOMER at 0x7f1b9b1da3e0>,
 <RNAP RNAP_CPLX8J236 at 0x7f1b9b1da6b0>,
 <RNAP RNAP_MONOMER8J26 at 0x7f1b9b1da590>}
```

Sinks to be curated through manual curation

The Troubleshooter finds gaps that need to be curated to allow for growth. Most of these are cofactors that will require some synthesis pathways (e.g. 4fe4s), but others will only need some transporters (e.g. metal ions). This was run with curated files, so the following is empty.

[16]:
```
me.reactions.query("^TS_")
```

[16]:
```
[]
```

2.8 Inspecting predicted fluxes

In this tutorial we will load and inspect the fluxes predicted by the *Bacillus subtilis* ME-model.

2.8.1 Import libraries

[1]:
```
from coralme.builder.main import MEBuilder
from coralme.util.flux_analysis import flux_based_reactions
import coralme
import pandas
import tqdm
```

2.8.2 Load

Load the ME-model coming out of the Troubleshooter as a JSON

[2]:
```
me = coralme.io.json.load_json_me_model("./bsubtilis/MEModel-step3-bsubtilis-TS.json")
```

```
Adding Metabolites into the ME-model...                                    : 100.0%|| ␣
↪4630/ 4630 [00:00<00:00]
Adding ProcessData into the ME-model...                                    : 100.0%|| ␣
↪4752/ 4752 [00:00<00:00]
Adding Reactions into the ME-model...                                      : 100.0%|| ␣
↪7758/ 7758 [00:18<00:00]
Updating ME-model Reactions...                                             : 100.0%|| ␣
↪6369/ 6369 [00:25<00:00]
```

2.8.3 Solve

[3]:
```
me.optimize()
```

```
The MINOS and quad MINOS solvers are a courtesy of Prof Michael A. Saunders. Please cite␣
↪Ma, D., Yang, L., Fleming, R. et al. Reliable and efficient solution of genome-scale␣
↪models of Metabolism and macromolecular Expression. Sci Rep 7, 40863 (2017). https://
↪doi.org/10.1038/srep40863

Iteration        Solution to check      Solver Status
---------       ------------------      -------------
        1       1.4050280687025918      Not feasible
        2       0.7025140343512959      Not feasible
        3       0.3512570171756479      Not feasible
        4       0.1756285085878240      Not feasible
        5       0.0878142542939120      Optimal
```

(continues on next page)

(continued from previous page)

```
        6       0.1317213814408680      Not feasible
        7       0.1097678178673900      Not feasible
        8       0.0987910360806510      Optimal
        9       0.1042794269740205      Not feasible
       10       0.1015352315273357      Not feasible
       11       0.1001631338039934      Optimal
       12       0.1008491826656645      Optimal
       13       0.1011922070965001      Optimal
       14       0.1013637193119179      Not feasible
       15       0.1012779632042090      Optimal
       16       0.1013208412580635      Not feasible
       17       0.1012994022311363      Not feasible
       18       0.1012886827176726      Not feasible
       19       0.1012833229609408      Not feasible
       20       0.1012806430825749      Not feasible
       21       0.1012793031433920      Optimal
       22       0.1012799731129835      Not feasible
```

[3]:
```
True
```

2.8.4 Inspecting fluxes

Predicted fluxes

[4]:
```
fluxes = me.solution.to_frame()
fluxes
```

[4]:
```
                                              fluxes  reduced_costs
biomass_dilution                        1.012793e-01  -5.648980e-01
BSU15140-MONOMER_to_generic_16Sm4Cm1402  1.506864e-12  -1.792344e-33
RNA_BSU_rRNA_1_to_generic_16s_rRNAs      1.465206e-06  -1.448997e-32
RNA_BSU_rRNA_16_to_generic_16s_rRNAs     0.000000e+00  -6.326530e-02
RNA_BSU_rRNA_19_to_generic_16s_rRNAs     5.674343e-07  -1.849227e-32
...                                               ...            ...
translocation_BSU25290_Plasma_Membrane   3.013728e-12  -4.779545e-34
translocation_BSU27650_Plasma_Membrane   1.354157e-06   0.000000e+00
translocation_BSU33630_Plasma_Membrane   1.370728e-06   0.000000e+00
translocation_BSU35300_Plasma_Membrane   1.354988e-06   3.520845e-34
translocation_BSU41040_Plasma_Membrane   3.277003e-08   9.992851e-34

[7758 rows x 2 columns]
```

Biomass production

[5]:
```
fluxes[fluxes.index.str.contains("biomass")]
```

[5]:
```
                                    fluxes  reduced_costs
biomass_dilution                  0.101279  -5.648980e-01
protein_biomass_to_biomass        0.034953  -2.375644e-34
mRNA_biomass_to_biomass           0.000116  -1.199677e-34
tRNA_biomass_to_biomass           0.000700   0.000000e+00
rRNA_biomass_to_biomass           0.005100  -7.874817e-35
ncRNA_biomass_to_biomass          0.000000   7.391429e-35
```

(continues on next page)

(continued from previous page)

```
tmRNA_biomass_to_biomass             0.000000   7.391429e-35
DNA_biomass_to_biomass               0.006003   7.391429e-35
lipid_biomass_to_biomass             0.023282   7.391429e-35
constituent_biomass_to_biomass       0.011441   7.391429e-35
prosthetic_group_biomass_to_biomass  0.000023  -5.556140e-35
peptidoglycan_biomass_to_biomass     0.000000   7.391429e-35
biomass_constituent_demand           0.101279   1.554562e-02
```

Transcription rates

[6]:
```
fluxes[fluxes.index.str.contains("^transcription_")].head()
```

[6]:
```
                                                 fluxes  reduced_costs
transcription_TU8J2-1243_from_BSU25200-MONOMER  0.000000e+00  -6.246011e-33
transcription_TU8J2-569_from_BSU25200-MONOMER   2.160400e-08   7.605084e-32
transcription_TU8J2-1577_from_BSU25200-MONOMER  0.000000e+00  -6.986124e-32
transcription_TU8J2-257_from_BSU25200-MONOMER   0.000000e+00  -1.127560e+03
transcription_TU8J2-677_from_BSU25200-MONOMER   1.736594e-08  -1.447121e-32
```

Translation rates

[7]:
```
fluxes[fluxes.index.str.contains("^translation_")].head()
```

[7]:
```
                           fluxes  reduced_costs
translation_BSU00090  1.419685e-08  -1.712708e-32
translation_BSU00110  6.498762e-13  -6.375952e-33
translation_BSU00120  6.498762e-13  -6.911721e-33
translation_BSU00130  2.293786e-08   2.102613e-32
translation_BSU00140  5.327400e-11   9.486318e-33
```

2.8.5 Biomass profile

We can calculate the predicted biomass composition in this condition by using *flux_based_reactions* which outputs the mass balance of a metabolite

[8]:
```
flux_based_reactions(me,"biomass")
```

[8]:
```
                                     lb      ub  rxn_flux  met_flux  \
biomass_dilution                     mu      mu  0.101279 -0.101279
protein_biomass_to_biomass          0.0  1000.0  0.034953  0.054615
lipid_biomass_to_biomass            0.0  1000.0  0.023282  0.023282
constituent_biomass_to_biomass      0.0  1000.0  0.011441  0.011441
DNA_biomass_to_biomass              0.0  1000.0  0.006003  0.006003
rRNA_biomass_to_biomass             0.0  1000.0  0.005100  0.005100
tRNA_biomass_to_biomass             0.0  1000.0  0.000700  0.000700
mRNA_biomass_to_biomass             0.0  1000.0  0.000116  0.000116
prosthetic_group_biomass_to_biomass 0.0  1000.0  0.000023  0.000023
tmRNA_biomass_to_biomass            0.0  1000.0  0.000000  0.000000
ncRNA_biomass_to_biomass            0.0  1000.0  0.000000  0.000000
peptidoglycan_biomass_to_biomass    0.0  1000.0  0.000000  0.000000

                                                                reaction
```

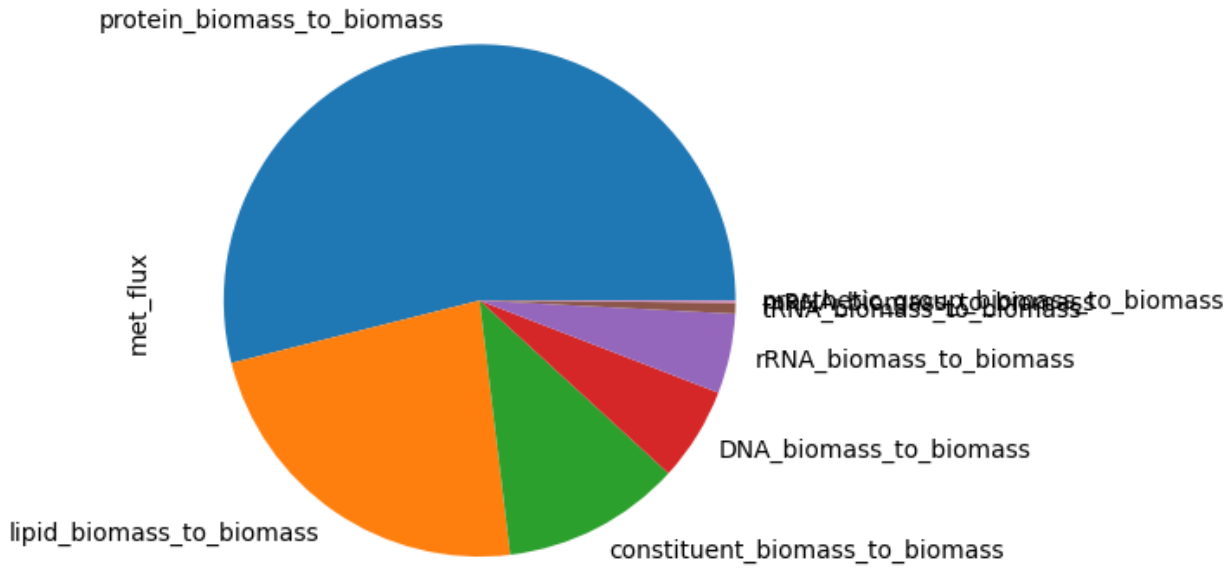
(continues on next page)

(continued from previous page)

```
biomass_dilution                                                1.0 biomass -->
protein_biomass_to_biomass          1.0 protein_biomass + 0.5625 unmodeled_protein...
lipid_biomass_to_biomass                          1.0 lipid_biomass --> 1.0 biomass
constituent_biomass_to_biomass               1.0 constituent_biomass --> 1.0 biomass
DNA_biomass_to_biomass                         1.0 DNA_biomass --> 1.0 biomass
rRNA_biomass_to_biomass                        1.0 rRNA_biomass --> 1.0 biomass
tRNA_biomass_to_biomass                        1.0 tRNA_biomass --> 1.0 biomass
mRNA_biomass_to_biomass                        1.0 mRNA_biomass --> 1.0 biomass
prosthetic_group_biomass_to_biomass      1.0 prosthetic_group_biomass --> 1.0 biomass
tmRNA_biomass_to_biomass                       1.0 tmRNA_biomass --> 1.0 biomass
ncRNA_biomass_to_biomass                       1.0 ncRNA_biomass --> 1.0 biomass
peptidoglycan_biomass_to_biomass            1.0 peptidoglycan_biomass --> 1.0 biomass
```

[9]:
```
tmp = flux_based_reactions(me,"biomass")["met_flux"]
BiomassComponents = tmp[tmp>0]
BiomassComponents.plot.pie()
```

[9]:
```
<Axes: ylabel='met_flux'>
```



2.9 Memory- and time-efficient solving of ME-models

In this tutorial we will convert the ME-model object to an NLP mathematical representation to save memory and time in simulating many conditions.

2.9.1 Import libraries

[1]:
```
import coralme
```

2.9.2 Load

Load the ME-model coming out of the Troubleshooter

[2]:
```
me = coralme.io.json.load_json_me_model("./bsubtilis/MEModel-step3-bsubtilis-TS.json")
```

```
Adding Metabolites into the ME-model...                                  : 100.0%|| ␣
↪4630/ 4630 [00:00<00:00]
Adding ProcessData into the ME-model...                                  : 100.0%|| ␣
↪4752/ 4752 [00:00<00:00]
Adding Reactions into the ME-model...                                    : 100.0%|| ␣
↪7758/ 7758 [00:18<00:00]
Updating ME-model Reactions...                                           : 100.0%|| ␣
↪6369/ 6369 [00:24<00:00]
```

2.9.3 Convert to NLP problem

The ME-model object *me* is a big object containing all data and metadata. This is not necessary when performing large-scale simulations, such as gene knockouts, or growth simulations under hundreds of conditions.

So, in these cases we only need the mathematical problem representing the ME-model, which is *nlp*.

[3]:
```
from coralme.solver.solver import ME_NLP
import cobra

def get_nlp(model):
    # Call construct LP problem function from model to get precursor objects.
    # lamdify = True -> Creates lambdify functions to calculate bounds as a function of␣
↪mu
    # per_position = True -> LB and UB bounds as list of lambdify instead of a lambdify␣
↪to
    # be able to change individual values
    Sf, Se, lb, ub, b, c, cs, atoms, lambdas, Lr, Lm = model.construct_lp_
↪problem(lambdify=True,per_position=True)

    # Construct NLP object from precursor objects
    me_nlp = ME_NLP(Sf, Se,b, c, lb, ub,  cs, atoms, lambdas)
    return me_nlp
```

[4]:
```
nlp = get_nlp(me)
```

2.9.4 Retrieve metabolite and reaction indexes

The *nlp* now contains the mathematical representation, very similar to a struct object of the COBRA Toolbox in MATLAB. Similarly, reactions and metabolites are now accessed from integer indexes. We can create a dictionary from the original model to map reaction ids to indexes

[5]:
```
rxn_index_dct = {r.id : me.reactions.index(r) for r in me.reactions}
met_index_dct = {m.id : me.metabolites.index(m) for m in me.metabolites}
```

From now on, *me* is no longer necessary and can be deleted to save memory usage. This is especially helpful when running parallelized simulations.

[6]:
```
# del me
```

2.9.5 Solving the MEModel vs. NLP

Now we can call the modified *optimize* function in *helpers*. This function was modified from the me.optimize() function of a coralme.core.model.MEModel.

Here you can see the speed-up when solving from scratch and solving from the NLP. The speed-up is even more noticeable with bigger models, as lamdifying a longer list of constraints will take much longer.

ME-model

[7]:
```
%%time
me.optimize(max_mu = 0.2, min_mu = 0., maxIter = 100, lambdify = True,
            tolerance = 1e-6, precision = 'quad', verbose = True)
```

```
The MINOS and quad MINOS solvers are a courtesy of Prof Michael A. Saunders. Please cite␣
↪Ma, D., Yang, L., Fleming, R. et al. Reliable and efficient solution of genome-scale␣
↪models of Metabolism and macromolecular Expression. Sci Rep 7, 40863 (2017). https://
↪doi.org/10.1038/srep40863

Iteration        Solution to check      Solver Status
---------       ------------------      -------------
        1       0.1000000000000000      Optimal
        2       0.1500000000000000      Not feasible
        3       0.1250000000000000      Not feasible
        4       0.1125000000000000      Not feasible
        5       0.1062500000000000      Not feasible
        6       0.1031250000000000      Not feasible
        7       0.1015625000000000      Not feasible
        8       0.1007812500000000      Optimal
        9       0.1011718750000000      Optimal
       10       0.1013671875000000      Not feasible
       11       0.1012695312500000      Optimal
       12       0.1013183593750000      Not feasible
       13       0.1012939453125000      Not feasible
       14       0.1012817382812500      Not feasible
       15       0.1012756347656250      Optimal
       16       0.1012786865234375      Optimal
       17       0.1012802124023438      Not feasible
       18       0.1012794494628906      Optimal
CPU times: user 1min 35s, sys: 36.7 ms, total: 1min 35s
Wall time: 1min 35s
```

[7]:
```
True
```

NLP

[8]:
```
def optimize(rxn_index_dct,met_index_dct,me_nlp,max_mu = 2.8100561374051836, min_mu = 0.,
↪ maxIter = 100,
            tolerance = 1e-6, precision = 'quad', verbose = True,basis=None):
    muopt, xopt, yopt, zopt, basis, stat = me_nlp.bisectmu(
                            mumax = max_mu,
                            mumin = min_mu,
                            maxIter = maxIter,
                            tolerance = tolerance,
                            precision = precision,
```

(continues on next page)

(continued from previous page)

```
                            verbose = verbose,
                basis=basis)

    if stat == 'optimal':
        #f = sum([ rxn.objective_coefficient * xopt[idx] for idx, rxn in enumerate(self.
↪reactions) ])
        x_primal = xopt[ 0:len(rxn_index_dct) ]   # The remainder are the slacks
        x_dict = { rxn : xopt[idx] for rxn,idx in rxn_index_dct.items() }
        #y = pi
        # J = [S; c]
        y_dict = { met : yopt[idx] for met,idx in met_index_dct.items() }
        z_dict = { rxn : zopt[idx] for rxn,idx in rxn_index_dct.items() }
        #y_dict['linear_objective'] = y[len(y)-1]

        #self.me.solution = Solution(f, x_primal, x_dict, y, y_dict, 'qminos', time_
↪elapsed, status)
        return cobra.core.Solution(
            objective_value = muopt,
            status = stat,
            fluxes = x_dict, # x_primal is a numpy.array with only fluxes info
            reduced_costs = z_dict,
            shadow_prices = y_dict,
            ),basis
    else:
        return None,None
```

[9]:
```
%%time
sol,basis = optimize(rxn_index_dct,met_index_dct,nlp,max_mu = 0.2, min_mu = 0., maxIter␣
↪= 100,
            tolerance = 1e-6, precision = 'quad', verbose = True, basis = None)
```

```
Iteration        Solution to check      Solver Status
---------       ------------------      -------------
        1       0.1000000000000000      Optimal
        2       0.1500000000000000      Not feasible
        3       0.1250000000000000      Not feasible
        4       0.1125000000000000      Not feasible
        5       0.1062500000000000      Not feasible
        6       0.1031250000000000      Not feasible
        7       0.1015625000000000      Not feasible
        8       0.1007812500000000      Optimal
        9       0.1011718750000000      Optimal
       10       0.1013671875000000      Not feasible
       11       0.1012695312500000      Optimal
       12       0.1013183593750000      Not feasible
       13       0.1012939453125000      Not feasible
       14       0.1012817382812500      Not feasible
       15       0.1012756347656250      Optimal
       16       0.1012786865234375      Optimal
       17       0.1012802124023438      Not feasible
       18       0.1012794494628906      Optimal
CPU times: user 1min 26s, sys: 36.2 ms, total: 1min 26s
```

(continues on next page)

(continued from previous page)

```
Wall time: 1min 26s
```

2.9.6 Re-using the basis for even more speed-up

We can re-use a basis from a previously successful simulation to warm-start the first iteration and save even more time!

Re-using basis

[10]:
```
%%time
sol,_ = optimize(rxn_index_dct,met_index_dct,nlp,max_mu = 0.2, min_mu = 0., maxIter = 1,
            tolerance = 1e-6, precision = 'quad', verbose = True, basis = basis)
```

```
Iteration        Solution to check      Solver Status
---------       ------------------      -------------
        1       0.1000000000000000      Optimal
CPU times: user 5.24 s, sys: 1 µs, total: 5.24 s
Wall time: 5.23 s
```

Cold start

[11]:
```
%%time
sol,_ = optimize(rxn_index_dct,met_index_dct,nlp,max_mu = 0.2, min_mu = 0., maxIter = 1,
            tolerance = 1e-6, precision = 'quad', verbose = True, basis = None)
```

```
Iteration        Solution to check      Solver Status
---------       ------------------      -------------
        1       0.1000000000000000      Optimal
CPU times: user 45.8 s, sys: 600 µs, total: 45.8 s
Wall time: 45.7 s
```

Full calculation

[12]:
```
%%time
sol,basis = optimize(rxn_index_dct,met_index_dct,nlp,max_mu = 0.2, min_mu = 0., maxIter␣
↪= 100,
            tolerance = 1e-6, precision = 'quad', verbose = True, basis = basis)
```

```
Iteration        Solution to check      Solver Status
---------       ------------------      -------------
        1       0.1000000000000000      Optimal
        2       0.1500000000000000      Not feasible
        3       0.1250000000000000      Not feasible
        4       0.1125000000000000      Not feasible
        5       0.1062500000000000      Not feasible
        6       0.1031250000000000      Not feasible
        7       0.1015625000000000      Not feasible
        8       0.1007812500000000      Optimal
        9       0.1011718750000000      Optimal
       10       0.1013671875000000      Not feasible
       11       0.1012695312500000      Optimal
       12       0.1013183593750000      Not feasible
       13       0.1012939453125000      Not feasible
```

(continues on next page)

(continued from previous page)

```
       14       0.1012817382812500      Not feasible
       15       0.1012756347656250      Optimal
       16       0.1012786865234375      Optimal
       17       0.1012802124023438      Not feasible
       18       0.1012794494628906      Optimal
CPU times: user 44.1 s, sys: 3.99 ms, total: 44.1 s
Wall time: 44.1 s
```

2.9.7 Modifying the NLP

As previously mentioned, the NLP resembles a struct object of the COBRA Toolbox. The model is stored as a collection of vectors and matrices representing stoichiometries, bounds and other variables needed by the solvers.

The relevant properties are:

- **xu**: Upper bounds
- **xl**: Lower bounds
- **S**: Stoichiometric matrix (Metabolites x Reactions)

The carbon source right now is Glucose, so we will change its bound to -10 to try to achieve maximum growth rate.

Note that bounds contain lambdify objects, not floats!

[13]:
```
nlp.xl[rxn_index_dct["EX_glc__D_e"]] = lambda x:-10
```

[14]:
```
%%time
sol,basis = optimize(rxn_index_dct,met_index_dct,nlp,max_mu = 0.5, min_mu = 0., maxIter␣
↪= 100,
            tolerance = 1e-6, precision = 'quad', verbose = True, basis = basis)
```

```
Iteration        Solution to check      Solver Status
---------       ------------------      -------------
        1       0.2500000000000000      Not feasible
        2       0.1250000000000000      Optimal
        3       0.1875000000000000      Optimal
        4       0.2187500000000000      Not feasible
        5       0.2031250000000000      Optimal
        6       0.2109375000000000      Not feasible
        7       0.2070312500000000      Optimal
        8       0.2089843750000000      Not feasible
        9       0.2080078125000000      Optimal
       10       0.2084960937500000      Optimal
       11       0.2087402343750000      Optimal
       12       0.2088623046875000      Not feasible
       13       0.2088012695312500      Not feasible
       14       0.2087707519531250      Optimal
       15       0.2087860107421875      Not feasible
       16       0.2087783813476562      Optimal
       17       0.2087821960449219      Optimal
       18       0.2087841033935547      Not feasible
       19       0.2087831497192383      Optimal
CPU times: user 29.8 s, sys: 11.7 ms, total: 29.8 s
Wall time: 29.8 s
```

2.9.8 Modifying the NLP from a dictionary of new bounds

Make sure to follow this method so that the lambda does not store pointers to a variable but rather a fixed constant (if that is what you want).

[15]:
```
def set_exchanges(nlp,dct):
    for k,v in dct.items():
        nlp.xl[rxn_index_dct[k]] = lambda _,x=v:x
```

[16]:
```
exchanges = {
    "EX_glc__D_e" : -10,
    "EX_o2_e" : -0.6,
    "EX_fru_e" : -5,
}
```

[17]:
```
set_exchanges(nlp,exchanges)
```

2.9.9 Inspecting the solution

The function returns a cobra.Solution object just like the one stored in me.solution. For more details inspecting *sol*, refer to *Inspecting predicted fluxes*.

[18]:
```
sol
```

[18]:
```
<Solution 0.209 at 0x7fbea76a0880>
```

2.10 Frequently Asked Questions

2.10.1 My gene identifiers are not consistent, what should I do?

We know that consistent gene ID conventions are a problem across all platforms of bioinformatics. We tried to generalize as much as possible what the gene conventions could be, but often different genome assemblies or M-model reconstructions yield inconsistent files.

What you should do depends on your problem, so we will classify the gene ID convention issues considering there are three main sources of information that must be consistent:

- **M-model** *gene identifiers*
- **Genome** *locus_tag*
- **Optional file** column *Accession-1*

Overall, you can assume that modifying the genome genbank file is the hardest approach and thus, the last resort.

1. M-model and Genbank are consistent, but they are not consistent with the BioCyc files.

Make sure that you looked for the correct BioCyc database, which corresponds to the M-model reconstruction. One quick way to ensure that is to copy one gene from your genbank or M-model and paste it in the search bar of BioCyc. Best case scenario, you microbe will appear in the list. Download the files from there and your problems are solved!

That didn't help?

It is possible that even when ensuring the BioCyc database is correct, the **Accession-1** column of genes.txt is still not consistent. However, you can assume that the correct IDs are somewhere in the database, since you found it looking for a gene id that follows your conventions (see *Getting Started*).

Try:

- Adding new columns in the gene **SmartTable**, **Accession-2** or **Synonyms** could contain your IDs.
- Maybe your IDs and BioCyc's only differ by an underscore, e.g. "PP0001" and "PP_0001". Use a text editor to change the IDs accordingly in **Accession-1** of genes.txt. Make sure not to make a mistake by editing gene IDs!

2. M-model and Genbank are not consistent

Make sure that you downloaded the same genbank file that was used to reconstruct the M-model, that is critical! If this is happening, you probably have the wrong genbank.

If you have a gene dictionary to convert between conventions, change the files to the IDs that are consistent with BioCyc.

CHAPTER

THREE

INDICES AND TABLES

- genindex
- modindex
- search

INDEX